

**ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - TIN HỌC**



**BÁO CÁO DỰ ÁN LẬP TRÌNH
PHẦN MỀM GIẢI BÀI TOÁN
QUY HOẠCH TUYẾN TÍNH**

Nhóm: Gold

Thành Viên Nhóm:

24110158 Lê Hoài Hậu
24110137 Nguyễn Quốc Đạt
24110149 Phạm Trần Khánh Duy
24110153 Phạm Ngọc Hải
24110191 Ngô Nguyễn Huy Khương

Thành phố Hồ Chí Minh, tháng 6 năm 2025

Mục lục

1	Giới thiệu	4
1.1	Lý do chọn đề tài	4
1.2	Mục tiêu đề tài	4
1.3	Công nghệ sử dụng	5
2	Cơ sở lý thuyết	6
2.1	Bài toán Quy hoạch tuyến tính	6
2.2	Thuật toán đơn hình	6
2.3	Thuật toán Bland và hiện tượng xoay vòng	7
2.4	Thuật toán hai pha	8
2.5	Các trường hợp đặc biệt trong Quy hoạch tuyến tính	9
2.5.1	Bài toán vô nghiệm	9
2.5.2	Bài toán không giới nội	9
2.5.3	Bài toán có vô số nghiệm tối ưu	9
2.5.4	Bài toán xoay vòng	9
3	Phân tích và Thiết kế phần mềm	10
3.1	Cấu trúc dữ liệu nền tảng	10
3.1.1	Cấu trúc quản lý miền biến (VarBound)	10
3.1.2	Cấu trúc đóng gói mô hình bài toán (LinearProgram)	10
3.1.3	Cấu trúc lưu trữ vết thuật toán (SimplexStep)	11
3.2	Kiến trúc hệ thống	12
3.2.1	Sơ đồ luồng hoạt động tổng thể	12
3.2.2	Hiện thực và cơ chế hoạt động của các hàm kiến trúc core	12
3.3	Thiết kế giao diện người dùng	15
3.3.1	Màn hình nhập liệu động	15
3.3.2	Màn hình hiển thị bảng kết quả lật trang (WdSolve)	16
3.3.3	Màn hình trực quan hóa hình học không gian (WdShowImage)	16
4	Cài đặt và hiện thực thuật toán	18
4.1	Kiến trúc tổng quan và cấu trúc dữ liệu nền tảng	18
4.2	Chi tiết cài đặt và cơ chế hoạt động của các hàm thành viên	18
4.2.1	Tiến trình chuẩn hóa bài toán	18
4.2.2	Xử lý ràng buộc miền biến số	20
4.2.3	Thiết lập ma trận cơ sở bằng khử Gauss-Jordan	21
4.2.4	Khởi tạo cấu trúc bảng đơn hình	23
4.2.5	Tìm kiếm biến vào	24
4.2.6	Tìm kiếm biến ra	25
4.2.7	Phép biến đổi khử toàn phần Gauss-Jordan	26
4.2.8	Vòng lặp thực thi tiến trình	27
4.2.9	Giải thuật toán Đơn hình Hai pha	28
4.2.10	Rà soát khả năng đa nghiệm	31
4.2.11	Tìm kiếm và xác thực điểm cực biên thứ hai	32
4.2.12	Trích xuất và tái cấu trúc nghiệm thực tế	34
4.3	Độ phức tạp thuật toán	35

4.3.1	Độ phức tạp không gian (Space Complexity)	35
4.3.2	Độ phức tạp thời gian (Time Complexity)	35
5	Thử nghiệm và đánh giá	37
5.1	Thử nghiệm với bài toán có nghiệm duy nhất	37
5.2	Thử nghiệm với bài toán vô nghiệm	38
5.3	Thử nghiệm với bài toán không giới nội	40
5.4	Thử nghiệm với bài toán vô số nghiệm	43
5.5	Thử nghiệm với bài toán xoay vòng	45
6	Tổng kết và hướng phát triển	48
6.1	Kết quả đạt được	48
6.2	Hạn chế và hướng phát triển	48
7	Tài liệu tham khảo	49
8	Hướng dẫn tải và cài đặt phần mềm	50
8.1	Giới thiệu chung	50
8.2	Quy trình cài đặt phần mềm	50
8.2.1	Tải phần mềm từ GitHub	50
8.2.2	Đối với hệ điều hành Windows	50
8.2.3	Đối với hệ điều hành MacOS	54
8.2.4	Đối với hệ điều hành Linux	56
8.3	Hướng dẫn cài đặt API Key cho chức năng Chatbot	59
8.3.1	Mở cửa sổ Hỏi/Đáp trong phần mềm	59
8.3.2	Dán API Key vào hộp thoại cài đặt	60
8.3.3	Truy cập Groq Console	60
8.3.4	Vào mục API Keys	61
8.3.5	Tạo API Key mới	61
8.3.6	Sao chép API Key	62
8.3.7	Dán API Key vào phần mềm	62
8.3.8	Kiểm tra chức năng Chatbot	62
8.4	Một số lỗi thường gặp khi cài API Key	62
8.4.1	Chatbot không phản hồi	62
8.4.2	Nhập /key nhưng không hiện hộp thoại	63
8.4.3	API Key bị lộ hoặc không còn sử dụng được	63
8.5	Hoàn tất cài đặt Chatbot	63
9	Hướng dẫn sử dụng	64
9.1	Hướng dẫn thao tác từng bước	64
9.1.1	Bước 1: Bắt đầu sử dụng	64
9.1.2	Bước 2: Khởi tạo kích thước bài toán	64
9.1.3	Bước 3: Nhập dữ liệu bài toán	65
9.1.4	Bước 4: Chọn phương pháp và giải bài toán	65
9.1.5	Bước 5: Hiển thị các bước giải và kết quả	66
9.1.6	Bước 6: Biểu diễn hình học cho bài toán 2 biến	67
9.1.7	Bước 7: Xem lời giải bằng file PDF	68
9.1.8	Bước 8: Chatbot hỗ trợ giải đáp	69

9.2	Xử lý các thông báo ngoại lệ	70
9.2.1	Trường hợp “Bài toán không giới nội”	70
9.2.2	Trường hợp “Vô nghiệm”	72
9.2.3	Trường hợp “Lỗi xoay vòng (Cycling)”	73
9.2.4	Trường hợp “Vô số nghiệm”	74
9.3	Các tính năng hỗ trợ nhập liệu nâng cao	76
10	Link mã nguồn của dự án và một vài mô tả về Repository	78
10.1	Link chứa toàn bộ mã nguồn của dự án	78
10.2	Ghi chú tổng quan về repository	78
10.3	Sơ đồ thư mục chính	78
10.4	Vai trò của từng nhóm thư mục/file	79
10.5	Ghi chú cho giảng viên khi xem mã nguồn	80
11	Một số phần mềm giải quy hoạch tuyến tính khác	81
12	Vai trò của các thành viên trong dự án	82

1. Giới thiệu

1.1 Lý do chọn đề tài

Trong bối cảnh khoa học công nghệ ngày càng phát triển, nhu cầu tối ưu hóa các hoạt động sản xuất, kinh doanh, vận tải, phân bổ nguồn lực và ra quyết định ngày càng trở nên quan trọng. Nhiều bài toán thực tiễn đòi hỏi phải tìm ra phương án tối ưu nhằm giảm chi phí, tăng lợi nhuận hoặc sử dụng hiệu quả các nguồn lực có hạn. Một trong những công cụ toán học mạnh mẽ và được ứng dụng rộng rãi để giải quyết các vấn đề đó là Quy hoạch tuyến tính.

Quy hoạch tuyến tính là một lĩnh vực quan trọng của Tối ưu hóa toán học, cho phép mô hình hóa các bài toán thực tế dưới dạng hàm mục tiêu tuyến tính cùng hệ các ràng buộc tuyến tính. Từ khi được phát triển bởi George Dantzig với thuật toán đơn hình, quy hoạch tuyến tính đã trở thành nền tảng của nhiều hệ thống hỗ trợ ra quyết định trong kinh tế, quản trị, logistics, nghiên cứu vận hành, khoa học dữ liệu và trí tuệ nhân tạo.

Trong quá trình học tập môn Quy hoạch tuyến tính, sinh viên thường phải thực hiện nhiều phép biến đổi thủ công như chuẩn hóa bài toán, lập bảng đơn hình, xác định biến vào – biến ra, thực hiện các phép xoay, xử lý các trường hợp đặc biệt như bài toán vô nghiệm, không giới nội, vô số nghiệm tối ưu hoặc áp dụng thuật toán hai pha đối với các bài toán không có nghiệm cơ sở ban đầu. Việc thực hiện bằng tay giúp hiểu rõ bản chất thuật toán nhưng lại tốn nhiều thời gian, dễ xảy ra sai sót trong quá trình tính toán và khó theo dõi đối với các bài toán có kích thước lớn.

Xuất phát từ những lý do trên, nhóm lựa chọn đề tài “Xây dựng phần mềm giải bài toán Quy hoạch tuyến tính” nhằm hỗ trợ quá trình học tập, nghiên cứu và kiểm chứng kết quả. Phần mềm không chỉ tự động thực hiện các bước giải mà còn trực quan hóa toàn bộ quá trình tính toán, giúp người dùng dễ dàng theo dõi từng bước của thuật toán và hiểu rõ hơn nguyên lý hoạt động của các phương pháp giải quy hoạch tuyến tính.

1.2 Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng một phần mềm hỗ trợ giải các bài toán quy hoạch tuyến tính một cách chính xác, trực quan và thuận tiện cho người sử dụng. Phần mềm được thiết kế nhằm tiếp nhận dữ liệu đầu vào của bài toán, tự động thực hiện các bước chuẩn hóa cần thiết và áp dụng các thuật toán phù hợp để tìm nghiệm tối ưu.

Bên cạnh việc cung cấp kết quả cuối cùng, hệ thống còn hướng đến khả năng trình bày chi tiết toàn bộ quá trình giải. Người dùng có thể theo dõi từng bước lựa chọn biến vào cơ sở, biến ra cơ sở, các phép xoay và sự thay đổi của từ vựng sau mỗi lần biến đổi. Ngoài ra, phần mềm còn hỗ trợ nhận biết các trường hợp đặc biệt như bài toán vô nghiệm, bài toán không giới nội, bài toán có vô số nghiệm tối ưu hoặc hiện tượng xoay vòng trong quá trình thực hiện thuật toán đơn hình.

Một mục tiêu quan trọng khác của đề tài là xây dựng giao diện trực quan giúp người học dễ dàng tương tác với chương trình. Đối với các bài toán có hai biến, phần mềm hỗ trợ biểu diễn hình học để người dùng quan sát được miền chấp nhận được và vị trí nghiệm tối ưu trên mặt phẳng tọa độ. Thông qua những chức năng này, phần

mềm không chỉ đóng vai trò là công cụ tính toán mà còn là phương tiện hỗ trợ học tập và nghiên cứu hiệu quả trong lĩnh vực quy hoạch tuyến tính. Bên cạnh đó, hệ thống còn được tích hợp chức năng chatbot hỗ trợ hỏi đáp, cho phép người dùng trao đổi và tìm hiểu các khái niệm, định lý, thuật toán cũng như các vấn đề liên quan đến Quy hoạch tuyến tính. Chức năng này góp phần nâng cao khả năng tương tác giữa người dùng và phần mềm, đồng thời hỗ trợ hiệu quả cho quá trình học tập và nghiên cứu.

1.3 Công nghệ sử dụng

Phần mềm được phát triển bằng ngôn ngữ lập trình C++ kết hợp với Qt Framework. Việc lựa chọn C++ xuất phát từ khả năng xử lý tính toán hiệu quả, hỗ trợ tốt cho việc xây dựng các cấu trúc dữ liệu và cài đặt các thuật toán tối ưu hóa có độ phức tạp tương đối cao. Ngôn ngữ này cho phép chương trình thực hiện các phép tính ma trận, biến đổi từ vệt và xử lý các bước của thuật toán đơn hình một cách nhanh chóng và chính xác.

Bên cạnh đó, Qt Framework được sử dụng để xây dựng giao diện đồ họa cho phần mềm. Qt cung cấp nhiều thư viện mạnh mẽ hỗ trợ thiết kế giao diện trực quan, quản lý sự kiện và hiển thị dữ liệu dưới dạng bảng biểu hoặc đồ họa. Nhờ Qt, người dùng có thể dễ dàng nhập dữ liệu bài toán, theo dõi các bước giải và quan sát kết quả một cách thuận tiện. Sự kết hợp giữa C++ và Qt giúp phần mềm vừa đảm bảo hiệu năng xử lý cao vừa đáp ứng yêu cầu về tính trực quan và thân thiện trong quá trình sử dụng.

2. Cơ sở lý thuyết

2.1 Bài toán Quy hoạch tuyến tính

Quy hoạch tuyến tính là một ngành của tối ưu hóa toán học nghiên cứu các bài toán tìm giá trị lớn nhất hoặc nhỏ nhất của một hàm mục tiêu tuyến tính với các ràng buộc tuyến tính. Một bài toán quy hoạch tuyến tính tổng quát có thể được biểu diễn dưới dạng:

$$\min (\max) c^T x$$

Với ràng buộc bất đẳng thức, đẳng thức:

$$Ax \leq b \quad \text{hoặc} \quad Ax \geq b \quad \text{hoặc} \quad Ax = b$$

Và ràng buộc dấu:

$$x \geq 0 \quad \text{hoặc} \quad x \leq 0 \quad \text{hoặc} \quad x \text{ tự do}$$

Trong đó, x là vector biến, c là vector hệ số của hàm mục tiêu, A là ma trận hệ số ràng buộc bất đẳng thức/đẳng thức và b là vector hằng số.

Để áp dụng các thuật toán giải, đặc biệt là phương pháp đơn hình, bài toán cần được đưa về dạng chuẩn có dạng:

$$\begin{aligned} \min c^T x \\ Ax \leq b \\ x \geq 0 \end{aligned}$$

Quá trình chuẩn hóa bao gồm việc chuyển đổi các ràng buộc về cùng một dạng, bổ sung biến phụ để biến bất phương trình thành phương trình và xử lý các biến không thỏa điều kiện không âm. Đối với biến tự do, có thể biểu diễn dưới dạng hiệu của hai biến không âm:

$$x_j = x_j^+ - x_j^- \quad \text{với} \quad x_j^+, x_j^- \geq 0$$

Việc đưa bài toán về dạng chuẩn giúp xây dựng được phương án cơ sở ban đầu và tạo điều kiện thuận lợi cho việc áp dụng các thuật toán tối ưu.

2.2 Thuật toán đơn hình

Phương pháp đơn hình là thuật toán cơ bản và phổ biến nhất để giải các bài toán quy hoạch tuyến tính. Ý tưởng của thuật toán dựa trên tính chất hình học rằng nếu bài toán có nghiệm tối ưu thì sẽ tồn tại ít nhất một nghiệm tối ưu tại một đỉnh cực biên của miền chấp nhận được.

Sau khi đưa bài toán về dạng chuẩn, mỗi nghiệm cơ sở tương ứng với một đỉnh cực biên của miền nghiệm. Thuật toán đơn hình bắt đầu từ một nghiệm cơ sở chấp nhận được, sau đó thực hiện các phép biến đổi để di chuyển từ đỉnh này sang đỉnh khác theo hướng cải thiện giá trị hàm mục tiêu. Quá trình này tiếp tục cho đến khi đạt được nghiệm tối ưu.

Trong mỗi bước lặp, thuật toán cần xác định biến vào cơ sở và biến ra khỏi cơ sở. Đối với quy tắc Dantzig, biến vào được chọn dựa trên hệ số âm nhất của hàm mục tiêu. Biến ra khỏi cơ sở được xác định bằng phép thử tỉ lệ nhỏ nhất, tức là trên cột chứa biến vào ta xét những dòng có hệ số a_{ik} dương và tính các tỉ lệ $\left| \frac{b_i}{a_{ik}} \right|$.

Sau đó ta xét min $\left| \frac{b_i}{a_{ik}} \right|$. Khi đó ta chọn biến ra ứng với dòng chứa min $\left| \frac{b_i}{a_{ik}} \right|$ làm biến ra cơ sở.

Sau khi xác định được biến vào và biến ra, thuật toán thực hiện phép xoay. Đây là phép biến đổi đại số nhằm cập nhật hệ phương trình để biến vào trở thành biến cơ sở mới và biến ra trở thành biến phi cơ sở. Từ vệt mới thu được vẫn biểu diễn cùng một bài toán nhưng tại một nghiệm cơ sở khác có giá trị hàm mục tiêu tốt hơn hoặc không kém nghiệm trước đó.

Thuật toán dừng khi không còn khả năng cải thiện giá trị hàm mục tiêu. Khi đó từ vệt hiện tại là từ vệt tối ưu và nghiệm cơ sở tương ứng chính là nghiệm tối ưu của bài toán.

2.3 Thuật toán Bland và hiện tượng xoay vòng

Trong quá trình thực hiện thuật toán đơn hình, việc lựa chọn biến vào cơ sở và biến ra khỏi cơ sở có vai trò quyết định đến tốc độ giải và tính ổn định của thuật toán. Thông thường, biến vào cơ sở được lựa chọn theo quy tắc Dantzig nhằm cải thiện giá trị hàm mục tiêu nhanh nhất. Tuy nhiên, đối với một số bài toán đặc biệt có nghiệm cơ sở suy biến, việc áp dụng quy tắc này có thể dẫn đến hiện tượng xoay vòng.

Hiện tượng xoay vòng là trường hợp thuật toán đơn hình sau một số bước biến đổi lại quay trở về một cơ sở đã xuất hiện trước đó. Khi điều này xảy ra, thuật toán tiếp tục lặp lại cùng một chuỗi các cơ sở mà không đạt được nghiệm tối ưu. Nguyên nhân của hiện tượng xoay vòng xuất phát từ các phép xoay suy biến, tức là các phép xoay làm thay đổi cơ sở nhưng không làm thay đổi nghiệm cơ sở và giá trị của hàm mục tiêu. Mặc dù hiện tượng này hiếm khi xuất hiện trong thực tế, nhưng về mặt lý thuyết nó có thể khiến thuật toán đơn hình không kết thúc sau hữu hạn bước.

Để khắc phục hiện tượng xoay vòng, một trong những phương pháp được sử dụng phổ biến là quy tắc Bland. Theo quy tắc này, khi có nhiều biến thỏa điều kiện được chọn vào cơ sở, ta ưu tiên chọn biến chính trước rồi mới chọn biến phụ sau, biến có chỉ số nhỏ nhất sẽ được ưu tiên lựa chọn. Tương tự, khi xác định biến ra khỏi cơ sở, biến cơ sở có chỉ số nhỏ nhất trong số các biến cũng được chọn. Việc lựa chọn theo thứ tự chỉ số giúp loại bỏ khả năng hình thành các chu trình lặp giữa các cơ sở. Định lý Bland khẳng định rằng nếu quy tắc này được áp dụng trong toàn bộ quá trình thực hiện thuật toán đơn hình thì hiện tượng xoay vòng sẽ không xảy ra và thuật toán chắc chắn kết thúc sau hữu hạn bước.

Ngoài quy tắc Bland, một phương pháp khác thường được sử dụng trong lý thuyết quy hoạch tuyến tính là phương pháp nhiễu. Phương pháp nhiễu là một kỹ thuật được sử dụng để ngăn ngừa hiện tượng xoay vòng trong thuật toán đơn hình. Ý tưởng cơ bản của phương pháp là thay đổi rất nhỏ các hằng số ở vế phải của hệ ràng buộc bằng cách bổ sung các số dương có độ lớn rất nhỏ. Khi đó, bài toán suy biến được chuyển thành một bài toán không suy biến theo nghĩa từ điển, nhờ đó loại bỏ khả năng xuất hiện các phép xoay suy biến liên tiếp.

Cụ thể, nếu vector vế phải của hệ ràng buộc là

$$b = (b_1, b_2, \dots, b_m)^T$$

thì phương pháp nhiều thay vector này bằng

$$b(\varepsilon) = (b_1 + \varepsilon, b_2 + \varepsilon^2, \dots, b_m + \varepsilon^m)^T$$

Trong đó $0 < \varepsilon \ll 1$ (Dấu $\varepsilon \ll 1$ được hiểu là ε rất nhỏ so với 1. Ví dụ: 10^{-6} ; 10^{-7} ; 10^{-8} , ...)

$$\text{Do } \varepsilon > \varepsilon^2 > \dots > \varepsilon^m > 0$$

các phần tử của $b(\varepsilon)$ luôn dương và khác nhau. Nhờ vậy, mỗi phép xoay trong thuật toán đơn hình đều tạo ra một sự thay đổi thực sự của nghiệm cơ sở và giá trị của các biến cơ sở theo thứ tự từ điển. Điều này đảm bảo rằng thuật toán không thể quay trở lại một cơ sở đã xuất hiện trước đó, từ đó loại bỏ hiện tượng xoay vòng.

Trong phạm vi đề tài, phần mềm được xây dựng có hỗ trợ quy tắc Bland như một phương pháp lựa chọn biến nhằm đảm bảo tính ổn định của thuật toán và hạn chế nguy cơ xảy ra hiện tượng xoay vòng. Việc áp dụng quy tắc này giúp chương trình hoạt động chính xác hơn đối với các bài toán suy biến, đồng thời nâng cao độ tin cậy của kết quả tính toán.

Về mặt lý thuyết, phương pháp nhiều đảm bảo thuật toán đơn hình luôn kết thúc sau hữu hạn bước. Trong thực tế, khi sử dụng kiểu dữ liệu số thực trên máy tính, các sai số làm tròn rất nhỏ cũng có tác dụng tương tự như một phép nhiễu tự nhiên. Chính vì vậy, nhiều bài toán xoay vòng trong lý thuyết vẫn có thể được giải thành công trên máy tính mà không xuất hiện hiện tượng lặp vô hạn.

2.4 Thuật toán hai pha

Xét bài toán quy hoạch tuyến tính dạng chuẩn

$$\begin{aligned} \min \quad & c^T x \\ & Ax \leq b \\ & x \geq 0 \end{aligned}$$

Trong đó $b = (b_1, b_2, \dots, b_m)^T$. Khi tồn tại ít nhất một thành phần $b_i < 0$, việc xây dựng trực tiếp một nghiệm cơ sở chấp nhận được ban đầu trở nên khó khăn, do đó phương pháp đơn hình thông thường không thể áp dụng ngay. Trong trường hợp này, thuật toán hai pha được sử dụng để tìm nghiệm cơ sở xuất phát cho bài toán.

Ở pha thứ nhất, biến phụ x_0 được đưa vào để xây dựng một bài toán phụ. Mục tiêu của pha này là tối thiểu hóa biến phụ x_0 nhằm tìm một nghiệm cơ sở chấp nhận được của bài toán gốc. Nếu giá trị tối ưu của bài toán phụ x_0 bằng không thì tồn tại một nghiệm cơ sở chấp nhận được và thuật toán chuyển sang pha hai. Ngược lại, nếu giá trị tối ưu x_0 vẫn dương thì bài toán gốc vô nghiệm.

Ở pha thứ hai, các biến phụ được loại bỏ và hàm mục tiêu ban đầu được khôi phục. Từ nghiệm cơ sở thu được ở pha một, phương pháp đơn hình tiếp tục được áp dụng để tìm nghiệm tối ưu của bài toán gốc.

Thuật toán hai pha là một phương pháp tổng quát và hiệu quả trong việc giải các bài toán quy hoạch tuyến tính mà việc xác định nghiệm cơ sở chấp nhận được ban đầu gặp khó khăn.

2.5 Các trường hợp đặc biệt trong Quy hoạch tuyến tính

Trong quá trình giải bằng phương pháp đơn hình có thể xuất hiện một số trường hợp đặc biệt cần được nhận biết và xử lý.

2.5.1 Bài toán vô nghiệm

Bài toán được gọi là vô nghiệm khi không tồn tại điểm nào đồng thời thỏa mãn tất cả các ràng buộc. Trong thuật toán hai pha, trường hợp này được phát hiện khi giá trị tối ưu của pha một lớn hơn không. Khi đó miền chấp nhận được rỗng và bài toán không có nghiệm.

2.5.2 Bài toán không giới nội

Bài toán không giới nội xảy ra khi giá trị hàm mục tiêu có thể tăng vô hạn đối với bài toán max hoặc giảm vô hạn đối với bài toán min. Trong phương pháp đơn hình, hiện tượng này được nhận biết khi đã chọn được biến vào cơ sở nhưng không tồn tại biến ra cơ sở tương ứng. Điều đó cho thấy thuật toán có thể tiếp tục cải thiện giá trị hàm mục tiêu mà không gặp giới hạn nào từ các ràng buộc.

2.5.3 Bài toán có vô số nghiệm tối ưu

Một bài toán được gọi là có vô số nghiệm tối ưu khi tồn tại nhiều hơn một nghiệm chấp nhận được cùng đạt giá trị tối ưu của hàm mục tiêu. Trong phương pháp đơn hình, dấu hiệu nhận biết là tại từ vựng tối ưu vẫn còn ít nhất một biến phi cơ sở có hệ số bằng không. Khi đó có thể thực hiện thêm một phép xoay mà không làm thay đổi giá trị hàm mục tiêu.

Về mặt hình học, tập nghiệm tối ưu không còn là một điểm duy nhất mà là một đoạn thẳng (trong mặt phẳng) hoặc một mặt (trong không gian) của miền chấp nhận được.

2.5.4 Bài toán xoay vòng

Hiện tượng xoay vòng là trường hợp thuật toán đơn hình sau một số phép xoay lại quay trở về một cơ sở đã xuất hiện trước đó và tiếp tục lặp lại chu trình này. Nguyên nhân chủ yếu của hiện tượng xoay vòng là sự xuất hiện của các nghiệm cơ sở suy biến, làm cho cơ sở thay đổi nhưng nghiệm cơ sở và giá trị hàm mục tiêu không thay đổi.

Mặc dù hiện tượng xoay vòng ít xảy ra trong thực tế, nhưng nó có thể khiến thuật toán đơn hình không kết thúc sau hữu hạn bước. Để khắc phục vấn đề này, có thể sử dụng quy tắc Bland hoặc phương pháp nhiễu.

3. Phân tích và Thiết kế phần mềm

3.1 Cấu trúc dữ liệu nền tảng

Để ánh xạ một bài toán Quy hoạch tuyến tính (QHTT) từ mô hình toán học thuần túy vào không gian bộ nhớ máy tính, hệ thống định nghĩa ba cấu trúc dữ liệu cốt lõi có mối quan hệ chặt chẽ với nhau: **VarBound**, **LinearProgram**, và **SimplexStep**. Sự tường minh của các cấu trúc này quyết định hiệu năng của core tính toán cũng như tính linh hoạt khi kết xuất dữ liệu lên lớp giao diện UI/UX.

3.1.1 Cấu trúc quản lý miền biến (VarBound)

Trong toán học tối ưu, miền xác định của các biến số x_j không chỉ bó hẹp ở điều kiện không âm ($x_j \geq 0$) mà có thể mở rộng thành biến không dương ($x_j \leq 0$) hoặc biến tự do ($x_j \in \mathbb{R}$). Cấu trúc **VarBound** đóng vai trò lưu trữ các điều kiện biên này của từng biến gốc.

```
1 \begin{lstlisting}
2 struct VarBound {
3     QString sign;    // Ký hiệu rang buoc mien bien (VD: ">=",
4                     // "<=", "free")
5     double value;    // Gia tri bien chan (thuong khoi tao la
6                     // 0.0)
7     bool isFree;     // Co danh dau bien tu do (true neu bien
8                     // thuoc R)
9 };
10
```

- **sign**: Chuỗi ký tự chuẩn hóa để phân loại miền giá trị. Nhận các token toán học như ">=", "<=", hoặc "free".
- **value**: Giá trị ngưỡng biên dưới hoặc biên trên phục vụ cho việc dời trục hoặc biến đổi ẩn phụ.
- **isFree**: Cờ logic nhị phân. Khi mang giá trị **true**, hệ thống sẽ bỏ qua kiểm tra dấu biên chặn và kích hoạt cơ chế tách ẩn phụ hiệu hai biến không âm ($x_j = x_j^+ - x_j^-$).

3.1.2 Cấu trúc đóng gói mô hình bài toán (LinearProgram)

LinearProgram là một đối tượng truyền tải dữ liệu đại diện trọn vẹn cho toàn bộ thông tin đại số của một bài toán QHTT nguyên bản ở dạng tổng quát.

```
1 struct LinearProgram {
2     std::vector<double> c;    // Vector he so ham
3                               // muc tieu
4     std::vector<std::vector<double>> A; // Ma tran he so rang
5                               // buoc cong nghe
6     std::vector<QString> signs; // Vector ky hieu
7                               // rang buoc (<=, =, >=)
8 };
9
```

```
5      std::vector<double> b;                // Vector ve phai tai
        nguyen
6      double c_0;                        // He so tu do cua
        ham muc tieu
7      std::vector<VarBound> varBounds;    // Mang cau hinh mien
        rang buoc cua tung bien
8      bool isMaximize = true;            // Co xac dinh kieu
        toi uu (True: Max, False: Min)
9      int algoType = 3;                  // Cau hinh thuat
        toan (0: Chuan, 1: Bland, 2: 2 Pha, 3: Tu dong)
10 };
```

- c và c_0 : Biểu diễn hàm mục tiêu $f(x) = c_1x_1 + c_2x_2 + \dots + c_nx_n + c_0$. Ma trận hệ số được lưu bằng vector một chiều liên tục.
- A : Ma trận hệ số ràng buộc kích thước $m \times n$, được tổ chức dưới dạng mảng hai chiều động (`std::vector<std::vector<double>>`), cho phép truy xuất nhanh phần tử hàng/cột qua cặp chỉ số `A[row][col]`.
- `signs` và `b`: Định hình cấu trúc của hệ ràng buộc bất đẳng thức và đẳng thức. Dòng ràng buộc thứ i sẽ có toán tử so sánh nằm ở `signs[i]` và giá trị giới hạn nằm ở `b[i]`.

3.1.3 Cấu trúc lưu trữ vết thuật toán (SimplexStep)

Một yêu cầu quan trọng của phần mềm là khả năng minh họa sự phạm, hiển thị tường minh sự biến đổi của từng bảng đơn hình qua mỗi bước lặp. `SimplexStep` được thiết kế như một ảnh chụp trạng thái (Snapshot) lưu lại toàn bộ thông tin tĩnh và động của một từ vựng đơn hình.

```
1 struct SimplexStep {
2     QString stepName;                    // Ten buoc (
        VD: "Bang khoi tao", "Vong lap 1")
3     std::vector<std::vector<double>> matrix;    // Trạng thái
        toan bo bang don hinh tai buoc hien tai
4     int pivotRow = -1;                  // Chỉ số hàng
        xoay duoc chon (-1 neu chua co/toi uu)
5     int pivotCol = -1;                  // Chỉ số cột
        xoay duoc chon (-1 neu chua co/toi uu)
6     std::vector<int> currentBasicVars;    // Danh sách
        chi so cac bien co so hien hanh
7     std::vector<double> solution;        // Giá trị của
        cac bien tai buoc lap hien tai
8 };
```

- `matrix`: Bản sao của bảng đơn hình `tableau` tại thời điểm ghi nhận. Ma trận này chứa cả hệ số ràng buộc đã biến đổi, về phải cập nhật và dòng chi phí rút gọn.

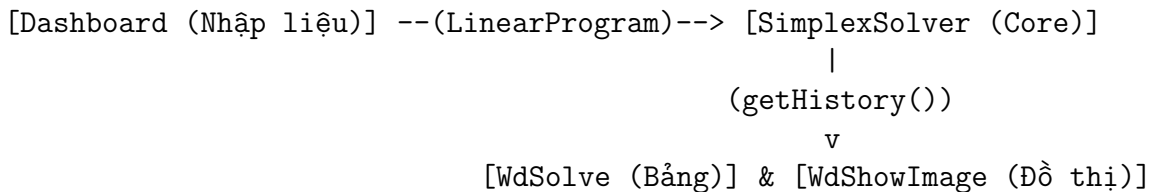
- `pivotRow` và `pivotCol`: Lưu tọa độ phần tử trục. Thông tin này vô cùng đắt giá cho lớp hiển thị UI, giúp hệ thống tô màu nổi bật ô trục xoay, hỗ trợ người dùng theo dõi logic chuyển đỉnh.
- `currentBasicVars` và `solution`: Ánh xạ nhanh bộ chỉ số biến đang nằm trong cơ sở và mảng nghiệm thực tế tương ứng, cho phép giao diện kết xuất thông tin nghiệm biên lập tức mà không phải tính toán lại.

3.2 Kiến trúc hệ thống

Phần mềm được xây dựng theo mô hình kiến trúc phân lớp, chia tách độc lập giữa tầng hiển thị, tầng điều phối logic và tầng xử lý toán học core.

3.2.1 Sơ đồ luồng hoạt động tổng thể

Tiến trình xử lý dữ liệu di chuyển tuyến tính theo một đường ống pipeline khép kín:



1. **Tầng nhập dữ liệu (Dashboard):** Người dùng tương tác cấu hình số biến, số ràng buộc. Lớp UI sinh động các trường nhập để đóng gói dữ liệu thô vào cấu trúc `LinearProgram`.
2. **Tầng xử lý trung tâm (SimplexSolver):** Nhận thực thể `LinearProgram`, kích hoạt bộ tiền xử lý chuẩn hóa, thiết lập bảng đơn hình đầu tiên và vận hành vòng lặp tính toán liên tục. Trạng thái mỗi vòng lặp được đẩy thẳng vào mảng lưu vết `std::vector<SimplexStep> history`.
3. **Tầng kết xuất đồ họa (WdSolve và WdShowImage):** `WdSolve` tiếp nhận mảng `history` để dựng giao diện bảng lật trang. Nếu số biến gốc bằng 2 hoặc 3, dữ liệu tọa độ các đỉnh trong `history` được gửi qua `WdShowImage` để vẽ hình học không gian vùng khả thi.

3.2.2 Hiện thực và cơ chế hoạt động của các hàm kiến trúc core

Hàm khởi tạo Core tính toán (`SimplexSolver::SimplexSolver`)

Hàm có nhiệm vụ tiếp nhận cấu hình bài toán từ giao diện nhập liệu, thiết lập trạng thái ban đầu cho các bộ đếm.

- Mã nguồn hiện thực:

```
1 SimplexSolver::SimplexSolver(const LinearProgram& Input) {
2     this->lp = Input;
3     this->alternativeOptimaFound = false;
4     this->iterationCount = 0;
5     this->statusMsg = "Khoi tao thanh cong";
6 }
```

- **Cơ chế hoạt động:** Hàm thực hiện sao chép sâu đối tượng Input vào biến thành viên ẩn `this->lp`. Toàn bộ cấu hình hệ số toán học của bài toán được cô lập bên trong thực thể Solver, ngắt kết nối với luồng luân chuyển dữ liệu ngoài UI để đảm bảo tính an toàn dữ liệu. Cờ kiểm soát đa nghiệm và bộ đếm vòng lặp được đưa về trạng thái mặc định.

Hàm ghi nhận vết thuật toán (SimplexSolver::recordStep)

Mỗi khi bảng đơn hình hoàn thành một phép xoay hàng toàn phần, trạng thái của nó phải được chụp lại trước khi bước lặp mới làm thay đổi giá trị cell bộ nhớ.

- **Mã nguồn hiện thực:**

```
1 void SimplexSolver::recordStep(const QString& name) {
2     SimplexStep step;
3     step.stepName = name;
4     step.matrix = this->tableau;
5     step.pivotRow = -1; // Toa do truc xoay se duoc cap nhat
6     step.pivotCol = -1; // dong o vong lap ngoai
7     step.currentBasicVars = this->basicVariables;
8     step.solution = this->getSolution(); // Trich xuat nghiem
9     // thuc te tai thoi diem hien tai
10    this->history.push_back(step);
11 }
```

- **Cơ chế hoạt động:** Hàm tạo ra một instance cục bộ của `SimplexStep`. Nó nạp chuỗi định danh bước lặp thông qua đối số `name` (Ví dụ: "Từ vựng 2"). Trạng thái vùng nhớ của ma trận tính toán bảng đơn hình (`this->tableau`) và mảng chỉ số cơ sở (`this->basicVariables`) được sao chép nguyên vẹn. Hàm gọi nội bộ đến `getSolution()` để giải mã ma trận cơ sở hiện tại thành mảng nghiệm thực tế của các biến ban đầu, giúp đóng gói một gói dữ liệu độc lập hoàn chỉnh trước khi đẩy ẩn vào mảng động `this->history`.

Hàm kích hoạt giải bài toán (SimplexSolver::solve)

Đây là hàm cửa ngõ điều phối tối cao của Core tính toán, kiểm soát toàn bộ vòng đời của thực thể giải thuật.

- **Mã nguồn hiện thực:**

```
1 bool SimplexSolver::solve() {
2     this->history.clear();
3     this->convertToStandardForm();
4
5     // Kiểm tra tính khả thi ban đầu để quyết định tiến trình
6     bool hasNegativeB = false;
7     for (double val : lp.b) {
8         if (val < -CLEAN_EPS) {
9             hasNegativeB = true;
10            break;
11        }
12    }
13
14    bool success = false;
15    if (hasNegativeB || lp.algoType == 2) {
16        success = this->solveTwoPhase();
17    } else {
18        this->buildTableau();
19        success = this->runSimplexLoop();
20        if (success) {
21            this->statusMsg = "Tối ưu";
22            this->firstSolution = this->getSolution();
23            this->altSolution = this->firstSolution;
24        }
25    }
26    return success;
27 }
```

- Cơ chế hoạt động:

1. Lệnh `history.clear()` làm sạch hoàn toàn bộ nhớ lưu vết từ các phiên giải trước đó.
2. Hành động `convertToStandardForm()` được gọi để xử lý miền biến và chuyển đổi bài toán về dạng chính tắc.
3. Thuật toán thực hiện một vòng quét trên vector tài nguyên về phải `lp.b`. Nếu tồn tại bất kỳ giá trị nào âm vượt ngưỡng `-CLEAN_EPS`, hệ thống kết luận nghiệm cơ sở ban đầu không khả thi. Cờ `hasNegativeB` bật lên ép hệ thống chuyển hướng sang hàm phức hợp `solveTwoPhase()` (Thuật toán 2 Pha với biến giả mở rộng).
4. Nếu về phải hoàn toàn dương, hệ thống tự tin khởi dựng bảng đơn hình chuẩn thông qua `buildTableau()` và kích hoạt `runSimplexLoop()`. Khi vòng lặp báo cấu trúc tối ưu đạt điều kiện dừng KKT, hàm tiến hành trích xuất mảng nghiệm lưu vào bộ đôi biến thành viên ẩn `firstSolution` và `altSolution`, hoàn thành chu trình giải với thông báo "Tối ưu".

3.3 Thiết kế giao diện người dùng

Hệ thống giao diện được thiết kế dựa trên thư viện đồ họa Qt Widgets, hướng tới trải nghiệm tường minh, nhất quán và đáp ứng linh hoạt theo số chiều biến của bài toán.

MAIN DASHBOARD		
Variables: [3]	Constraints: [2]	Generate Form
Objective Function: Min $Z = [3.0]x_1 + [-4.0]x_2 + [0.0]x_3 + [0.0]$		
Constraints Table: $[1.0]x_1 + [2.0]x_2 + [1.0]x_3$ $[\leq]$ $[4.0]$ $[3.0]x_1 + [0.0]x_2 + [2.0]x_3$ $[\geq]$ $[15.0]$		
Variable Bounds: $x_1 [\geq 0]$ $x_2 [\leq 0]$ $x_3 [\text{free}]$		
EXECUTE SOLVER		

Bảng 3.3.1: Mô phỏng giao diện Main Dashboard

3.3.1 Màn hình nhập liệu động

Màn hình này giải quyết bài toán biến đổi giao diện theo kích thước dữ liệu tùy chọn của người dùng.

- **Cơ chế dựng layout động:** Khi người dùng nhập số lượng ẩn số (n) và số lượng ràng buộc (m) rồi kích hoạt nút lệnh, lớp giao diện sẽ xóa bỏ các thành phần cũ trong Grid Layout và chạy vòng lặp khởi tạo các con trỏ `QLineEdit` (ô nhập số thực) và `QComboBox` (hộp chọn dấu ràng buộc).
- **Cấu trúc phân vùng tương tác:**
 - *Khu vực hàm mục tiêu:* Sinh ra một dãy ô nhập nằm ngang, xen kẽ là các nhãn hiển thị tên biến $x_1, x_2, \dots, x_n$ cùng với một nút gạt chuyển đổi chế độ toán học giữa **Maximize** và **Minimize**.
 - *Khu vực ma trận ràng buộc:* Một bảng lưới kích thước $m \times n$ của các ô `QLineEdit` đại diện cho hệ số công nghệ ma trận A . Cột kế cuối là một danh sách chọn chứa bộ ba toán tử Toán học ($\leq, =, \geq$), và cột cuối cùng thu nhận vector về phải b .
 - *Khu vực điều kiện biên:* Tạo ra n dòng tương ứng với n biến, cho phép người dùng quy định điều kiện dấu của từng ẩn số một cách độc lập.

3.3.2 Màn hình hiển thị bảng kết quả lặp trang (WdSolve)

Sau khi bấm lệnh giải bài toán, cửa sổ WdSolve được khởi tạo, chịu trách nhiệm kết xuất đồ họa dữ liệu từ mảng `std::vector<SimplexStep> history`.

STEP-BY-STEP TABLEAU VIEWER (WdSolve)						
Step: [Từ vệtng 2]			< Prev		Next >	
Basis	x1	x2	x3	s1	s2	RHS
x2	0.5	1.0	0.5	0.5	0.0	2.0
s2	3.0	0.0	2.0	0.0	1.0	15.0
-Z	2.0	0.0	2.0	2.0	0.0	8.0
Status: Running... Pivot Selected: Row 1, Col 0 (Val: 3.0)						

Bảng 3.3.2: Mô phỏng giao diện Tableau Viewer hiển thị từng bước lặp

- **Kiến trúc Widget trực quan:** Trung tâm màn hình sử dụng một đối tượng `QTableWidget` để tái hiện lại cấu trúc từ vệtng đơn hình. Hàng trên cùng đóng vai trò hiển thị tên của tập biến mở rộng (x_j , biến bù s_i , biến nhân tạo a_i). Cột đầu tiên hiển thị danh sách các biến đang nắm giữ ma trận cơ sở tại bước đó.
- **Cơ chế lặp trang điều hướng:** Phía trên bảng kết quả bố trí hai nút bấm điều hướng `< Prev` và `Next >` đi kèm với một thanh trượt tiến trình. Khi người dùng click chuyển trang, giao diện thay đổi chỉ số hiển thị bước lặp k , lập tức nạp cấu phần `history[k]` tương ứng, vẽ lại toàn bộ giá trị trên lưới `QTableWidget`.
- **Kỹ thuật tô màu phần tử trục (Pivot Highlighting):** Tại mỗi bước lặp, giao diện trích xuất hai chỉ số `pivotRow` và `pivotCol` từ cấu trúc dữ liệu bước. Nếu chỉ số hợp lệ (≥ 0), cell tại vị trí đó trong bảng lưới sẽ được hệ thống thiết lập bằng màu nền nổi bật (Ví dụ: màu vàng chanh hoặc cam nhạt, mô phỏng bằng khung viền đậm trong báo cáo) để người dùng nhận diện ngay phần tử trục xoay của bước lặp kế tiếp.

3.3.3 Màn hình trực quan hóa hình học không gian (WdShowImage)

Đối với các bài toán có số lượng biến gốc giới hạn trong không gian trực quan ($n = 2$ hoặc $n = 3$), hệ thống cung cấp tính năng mở rộng `WdShowImage` nhằm hình học hóa miền khả thi đa diện lồi.

- **Vẽ đồ thị 2D ($n = 2$):** Sử dụng thư viện đồ họa để dựng hệ trục tọa độ Ox_1x_2 . Các đường thẳng ràng buộc $a_{i1}x_1 + a_{i2}x_2 = b_i$ được vẽ trên mặt phẳng. Hệ thống sử dụng thuật toán tô đa giác để làm nổi bật miền khả thi lồi (Feasible Region). Từ dữ liệu mảng nghiệm `solution` trong từng bước của tập `history`, một đường mũi tên đồ thị màu đỏ được vẽ nối liền các đỉnh cực biên với nhau, mô tả chính xác quỹ đạo chuyển động di chuyển đỉnh của thuật toán Đơn hình trong không gian hai chiều.

- **Trực quan hóa 3D ($n = 3$):** Tận dụng module đồ họa không gian ba chiều để vẽ hệ trục tọa độ $Ox_1x_2x_3$. Các ràng buộc lúc này trở thành các mặt phẳng cắt nhau trong không gian. Phần mềm dựng lên một khối đa diện lồi đại diện cho không gian nghiệm khả thi. Quá trình giải toán được mô phỏng sinh động bằng một điểm cầu sáng nhỏ di chuyển dọc theo các cạnh của khối đa diện lồi từ đỉnh xuất phát ban đầu, nhảy qua các đỉnh trung gian, và dừng lại ở đỉnh tối ưu toàn cục, mang lại trải nghiệm UX trực quan sinh động và có chiều sâu học thuật cho người sử dụng.

4. Cài đặt và hiện thực thuật toán

4.1 Kiến trúc tổng quan và cấu trúc dữ liệu nền tảng

Để hiện thực hóa phương pháp Đơn hình (Simplex Method) một cách hướng đối tượng và đảm bảo tính chặt chẽ về mặt toán học, hệ thống được phân tách thành các cấu trúc dữ liệu tuyến tính cơ bản (Struct.h) và lớp xử lý cốt lõi SimplexSolver (simplexsolver.h).

Trước khi đi vào chi tiết các hàm kiểm soát tiến trình, ta xét cấu trúc mô hình hóa bài toán Quy hoạch tuyến tính:

```
1 struct VarBound {
2     QString sign;
3     double value;
4     bool isFree;
5 };
6
7 struct LinearProgram {
8     std::vector<double> c;
9     std::vector<std::vector<double>> A;
10    std::vector<QString> signs;
11    std::vector<double> b;
12    double c_0;
13    std::vector<VarBound> varBounds;
14    bool isMaximize = true;
15    // 0: Đơn hình chuẩn, 1: Bland, 2: 2 Pha, 3: Tự động
16    int algoType = 3;
17 };
```

- **VarBound**: Mô hình hóa miền ràng buộc của từng biến số x_j , phân tách rõ các trạng thái biến tự do (`isFree`) hoặc dấu ràng buộc (`sign`).
- **LinearProgram**: Đóng gói toàn bộ các thành phần đại số của một bài toán QHTT: vector hệ số hàm mục tiêu $c \in \mathbb{R}^n$, ma trận hệ số ràng buộc $A \in \mathbb{R}^{m \times n}$, vector vế phải - hằng số $b \in \mathbb{R}^m$, hệ số tự do c_0 , và các cờ cấu hình giải thuật (`isMaximize`, `algoType`).

4.2 Chi tiết cài đặt và cơ chế hoạt động của các hàm thành viên

4.2.1 Tiến trình chuẩn hóa bài toán

Hàm `convertToStandardForm` đóng vai trò là hàm điều phối tối cao trong giai đoạn tiền xử lý. Mục tiêu là đưa bài toán QHTT bất kỳ về dạng chuẩn toán học:

$$\begin{aligned}\min Z &= c^T x + c_0, \\ Ax &= b, \\ x &\geq 0\end{aligned}$$

Code thực hiện:

```
1 void SimplexSolver::convertToStandardForm() {
2     this->originalVarsCount = lp.c.size();
3     this->originalVarBounds = lp.varBounds;
4     if (lp.isMaximize) {
5         for (size_t i = 0; i < lp.c.size(); ++i) {
6             lp.c[i] = -lp.c[i];
7         }
8         lp.c_0 = -lp.c_0;
9     }
10
11     for (int i = 0; i < (int)lp.signs.size(); ++i) {
12         if (lp.signs[i] == ">=") {
13             lp.signs[i] = "<=";
14             lp.b[i] = -lp.b[i];
15             for (int j = 0; j < (int)lp.A[i].size(); ++j)
16                 lp.A[i][j] = -lp.A[i][j];
17         }
18     }
19
20     this->handleVariableBounds();
21     this->addSlackAndSurplusVariables();
22 }
```

Cơ chế hoạt động:

- Sao lưu thông tin gốc: Hàm ghi nhận số lượng biến nguyên bản thông qua `originalVarsCount` và sao chép mảng điều kiện biên `originalVarBounds` để phục vụ công tác phục hồi nghiệm thực tế sau khi giải xong.
- Chuyển đổi bài toán Max về Min: Đại số tuyến tính chứng minh rằng việc tìm cực đại của hàm mục tiêu tương đương với việc tìm cực tiểu của hàm đối ngẫu đảo dấu. Do đó, nếu `lp.isMaximize` mang giá trị true, vòng lặp sẽ đảo dấu từng phần tử của vector `lp.c` và hằng số `lp.c_0`.
- Chuẩn hóa chiều của bất đẳng thức: Quét qua mảng ký hiệu ràng buộc `lp.signs`. Nếu phát hiện ràng buộc dạng $\geq$, thuật toán nhân cả hai vế với -1 . Thao tác này đòi hỏi đảo dấu cấu phần vế phải `lp.b[i]` và toàn bộ các hệ số công nghệ trên dòng đó thuộc ma trận `lp.A[i][j]`. Ký hiệu ràng buộc được cập nhật thành $\leq$.
- Lời gọi hàm phụ thuộc: Cuối cùng, hàm tuần tự kích hoạt `handleVariableBounds()` để đồng nhất miền biến, và `addSlackAndSurplusVariables()` để chèn các biến bù/phụ.

4.2.2 Xử lý ràng buộc miền biến số

Hàm `handleVariableBounds` giải quyết các điều kiện biên phi chuẩn của biến số nhằm đưa toàn bộ hệ thống biến về điều kiện không âm ($x_j \geq 0$).

Code thực hiện:

```
1 void SimplexSolver::handleVariableBounds() {
2     for (int j = (int)lp.varBounds.size() - 1; j >= 0; --j) {
3         VarBound vb = lp.varBounds[j];
4         if (vb.isFree || vb.sign == "free") {
5             for (int i = 0; i < (int)lp.A.size(); ++i)
6                 lp.A[i].insert(lp.A[i].begin() + j + 1, -lp.A[i][j]);
7             lp.c.insert(lp.c.begin() + j + 1, -lp.c[j]);
8             lp.varBounds[j].sign = ">=";
9             lp.varBounds.insert(lp.varBounds.begin() + j + 1, {
10                 ">=", 0.0, false});
11             originalVarsCount++;
12         }
13         else if (vb.sign == "<=") {
14             for (int i = 0; i < (int)lp.A.size(); ++i)
15                 lp.A[i][j] = -lp.A[i][j];
16             lp.c[j] = -lp.c[j];
17             lp.varBounds[j].sign = ">=";
18         }
19     }
```

Phân tích cơ chế hoạt động:

- Chiến lược duyệt ngược: Vòng lặp bắt đầu từ chỉ số cuối cùng `(int)lp.varBounds.size() - 1` chạy lùi về 0. Kỹ thuật này là bắt buộc trong cấu trúc dữ liệu mảng động (`std::vector`). Khi ta chèn phần tử mới vào vị trí $j + 1$, các phần tử phía sau sẽ bị đẩy sang phải. Nếu duyệt tiến (từ 0 đến cuối), việc chèn phần tử sẽ làm thay đổi chỉ số của các cấu phần chưa xét, dẫn đến lỗi logic nghiêm trọng hoặc lặp vô hạn. Duyệt ngược đảm bảo các vị trí phía trước không bị ảnh hưởng bởi phép chèn phía sau.
- Cơ chế tách biến tự do: Đối với một biến tự do $x_j \in \mathbb{R}$, ta áp dụng phép thế toán học: $x_j = x_j^+ - x_j^-$ với $x_j^+, x_j^- \geq 0$. Trong mã nguồn, x_j^+ sẽ kế thừa vị trí chỉ số j hiện tại, còn x_j^- được đại diện bởi một cột mới chèn vào vị trí $j + 1$. Hệ số của biến mới trong ma trận ràng buộc và hàm mục tiêu chính là giá trị âm của hệ số biến cũ, được thể hiện qua các dòng code:

```
1 lp.A[i].insert(lp.A[i].begin() + j + 1, -lp.A[i][j]);
2 lp.c.insert(lp.c.begin() + j + 1, -lp.c[j]);
```

Đồng thời, cấu trúc ràng buộc biên được cập nhật lại thành ≥ 0 cho cả hai biến mới, và biến đếm `originalVarsCount` tăng lên để ghi nhận sự xuất hiện của cấu phần biến mở rộng này.

- Cơ chế đổi dấu biến âm: Nếu biến số được định nghĩa $x_j \leq 0$, thuật toán thực hiện phép đặt ẩn phụ $x'_j = -x_j \Rightarrow x'_j \geq 0$. Khi đó, mọi hệ số liên quan đến x_j đều phải đổi dấu: cột thứ j trong ma trận ràng buộc A nhân với -1 ($lp.A[i][j] = -lp.A[i][j]$) và hệ số chỉ phí hàm mục tiêu đổi dấu ($lp.c[j] = -lp.c[j]$).

4.2.3 Thiết lập ma trận cơ sở bằng khử Gauss-Jordan

Hàm `addSlackAndSurplusVariables` thực hiện quá trình đẳng thức hóa hệ ràng buộc ban đầu. Nhằm bảo toàn nghiêm ngặt cấu trúc hình học của đa diện nghiệm, giải thuật không sử dụng kỹ thuật cấp phát biến nhân tạo (artificial variables) cho các ràng buộc mang dấu $=$. Thay vào đó, hệ thống chủ động vận dụng các phép biến đổi sơ cấp hàng (Gauss-Jordan elimination) để kiến thiết ma trận cơ sở, qua đó loại trừ rủi ro nói lỏng miền khả thi (relaxation) làm sai lệch kết quả.

Code thực hiện:

```
1 void SimplexSolver::addSlackAndSurplusVariables() {
2     int m = (int)lp.signs.size();
3     this->basicVariables.assign(m, -1);
4
5     for (int i = 0; i < m; ++i) {
6         QString s = lp.signs[i].trimmed();
7         if (s == "<=") {
8             for (int row = 0; row < m; ++row)
9                 lp.A[row].push_back((row == i) ? 1.0 : 0.0);
10            lp.c.push_back(0.0);
11            basicVariables[i] = (int)lp.c.size() - 1;
12            lp.signs[i] = "=";
13        } else if (s == "=" || s == ">=") {
14            lp.signs[i] = "=";
15        }
16    }
17    std::set<int> usedBasicCols;
18    for (int i = 0; i < m; ++i) {
19        if (basicVariables[i] >= 0) usedBasicCols.insert(
20            basicVariables[i]);
21    }
22    for (int i = 0; i < m; ++i) {
23        if (basicVariables[i] != -1) continue;
24        for (int j = 0; j < (int)lp.c.size(); ++j) {
25            if (usedBasicCols.count(j)) continue;
26            if (std::abs(lp.A[i][j] - 1.0) > CLEAN_EPS)
27                continue;
28
29            bool isIdentity = true;
30            for (int r = 0; r < m; ++r) {
31                if (r != i && std::abs(lp.A[r][j]) > CLEAN_EPS)
32                    {
33                        isIdentity = false;
34                        break;
35                    }
36            }
37            if (!isIdentity) continue;
38            for (int row = 0; row < m; ++row)
39                lp.A[row][j] *= -1;
40            lp.c[j] *= -1;
41            basicVariables[i] = j;
42        }
43    }
44 }
```

```
33         }
34     }
35     if (isIdentity) {
36         basicVariables[i] = j;
37         usedBasicCols.insert(j);
38         break;
39     }
40 }
41 }
42 for (int i = 0; i < m; ++i) {
43     if (basicVariables[i] != -1) continue;
44     int pivotCol = -1;
45     double bestAbs = 0.0;
46
47     for (int j = 0; j < (int)lp.c.size(); ++j) {
48         if (usedBasicCols.count(j)) continue;
49         double v = std::abs(lp.A[i][j]);
50         if (v > bestAbs + CLEAN_EPS) {
51             bestAbs = v;
52             pivotCol = j;
53         }
54     }
55
56     if (pivotCol == -1 || bestAbs <= CLEAN_EPS) continue;
57
58     double pivot = lp.A[i][pivotCol];
59     for (int j = 0; j < (int)lp.c.size(); ++j) {
60         lp.A[i][j] /= pivot;
61         if (std::abs(lp.A[i][j]) < CLEAN_EPS) lp.A[i][j] =
62             0.0;
63     }
64     lp.b[i] /= pivot;
65     if (std::abs(lp.b[i]) < CLEAN_EPS) lp.b[i] = 0.0;
66
67     for (int r = 0; r < m; ++r) {
68         if (r == i) continue;
69         double factor = lp.A[r][pivotCol];
70         if (std::abs(factor) <= CLEAN_EPS) continue;
71         for (int j = 0; j < (int)lp.c.size(); ++j) {
72             lp.A[r][j] -= factor * lp.A[i][j];
73             if (std::abs(lp.A[r][j]) < CLEAN_EPS) lp.A[r][j] =
74                 0.0;
75         }
76         lp.b[r] -= factor * lp.b[i];
77         if (std::abs(lp.b[r]) < CLEAN_EPS) lp.b[r] = 0.0;
78     }
79
80     basicVariables[i] = pivotCol;
81     usedBasicCols.insert(pivotCol);
82 }
```

81 }

Phân tích cơ chế hoạt động:

- **Bước 1 (Chèn biến bù có chọn lọc):** Thuật toán chỉ thêm biến bù (+1) cho các ràng buộc có dấu $\leq$. Ràng buộc = được giữ nguyên bản chất phương trình, tránh việc bài toán bị rơi lỏng thành $\leq$ gây sai lệch tập nghiệm.
- **Bước 2 (Bảo tồn cột đơn vị tự nhiên):** Trước khi tác động biến đổi đại số, hệ thống quét ma trận để tìm các cột hiện tại đã có cấu trúc vector đơn vị e_i . Nếu có, chúng được nạp ngay vào tập `usedBasicCols` làm biến cơ sở tự nhiên.
- **Bước 3 (Cơ sở hóa bằng khử Gauss-Jordan):** Với các dòng đẳng thức chưa có cơ sở, thuật toán tìm phần tử có trị tuyệt đối lớn nhất trên dòng (Partial Pivoting) để làm trục xoay nhằm giảm thiểu sai số làm tròn. Sau đó, nó thực hiện phép chia tỷ lệ dòng trục, và trừ đi bội số tương ứng ở các dòng khác để triệt tiêu mọi hệ số trên cột `pivotCol` về 0. Thao tác này cưỡng bức biến x_j trở thành biến cơ sở hợp lệ mà không cần sử dụng đến biến giả.

4.2.4 Khởi tạo cấu trúc bảng đơn hình

Hàm `buildTableau` chuyển đổi toàn bộ dữ liệu đại số dạng mảng tuyến tính riêng rẽ thành một cấu trúc ma trận hai chiều duy nhất, tối ưu hóa vùng đệm nhớ cho các phép khử Gauss-Jordan kế tiếp.

Code thực hiện:

```

1 void SimplexSolver::buildTableau() {
2     int m = lp.A.size();
3     int n = lp.c.size();
4     tableau.assign(m + 1, std::vector<double>(n + 1, 0.0));
5
6     for (int i = 0; i < m; ++i) {
7         for (int j = 0; j < n; ++j) tableau[i][j] = lp.A[i][j];
8         tableau[i][n] = lp.b[i];
9     }
10    for (int j = 0; j < n; ++j)
11        tableau[m][j] = lp.c[j];
12    iterationCount = 0;
13    QString stepTitle = lp.isMaximize ? "Tu vung 1 (Bien doi
        Max Z -> Min -Z)" : "Tu vung 1 (Khoi tao)";
14    recordStep(stepTitle);
15 }

```

Cơ chế hoạt động:

- Thiết lập kích thước bảng: Ma trận `tableau` được phân bổ vùng nhớ động với kích thước $(m+1) \times (n+1)$. Dòng cộng thêm m đại diện cho dòng chỉ phí rút gọn của hàm mục tiêu (dòng chỉ số tối ưu), và cột cộng thêm n đại diện cho vector về phải chứa các giá trị nghiệm hiện hành b_i .
- Ánh xạ dữ liệu đại số: Khối $m \times n$ đầu tiên nhận giá trị từ ma trận hệ số ràng buộc: `tableau[i][j] = lp.A[i][j]`.

- + Cột cuối cùng (chỉ số n) nhận các giá trị hệ số b_i về phải: `tableau[i][n] = lp.b[i]`.
- + Dòng cuối cùng (chỉ số m) lưu trữ trực tiếp vector hệ số chi phí: `tableau[m][j] = lp.c[j]`. Phần tử góc đáy bên phải `tableau[m][n]` ban đầu bằng 0.0, đóng vai trò lưu trữ giá trị tối ưu của hàm mục tiêu sau này.
- Ghi nhận trạng thái ban đầu: Reset biến đếm bước lặp `iterationCount = 0` và gọi hàm `recordStep` để chụp lại ảnh ma trận đầu tiên đưa vào bộ lưu trữ lịch sử.

4.2.5 Tìm kiếm biến vào

Hàm `findPivotColumn` phân tích dòng kiểm tra tối ưu (dòng mục tiêu) để quyết định biến phi cơ sở nào sẽ được đưa vào làm biến cơ sở nhằm giảm giá trị hàm mục tiêu.

Code thực hiện:

```
1 int SimplexSolver::findPivotColumn() {
2     int m = tableau.size() - 1;
3     int n = tableau[0].size() - 1;
4     if (lp.algoType == 1) {
5         for (int j = 0; j < n; ++j)
6             if (tableau[m][j] < -CLEAN_EPS) return j;
7     } else {
8         double minVal = -CLEAN_EPS;
9         int bestCol = -1;
10        for (int j = 0; j < n; ++j) {
11            if (tableau[m][j] < minVal) {
12                minVal = tableau[m][j];
13                bestCol = j;
14            }
15        }
16        return bestCol;
17    }
18    return -1;
19 }
```

Cơ chế hoạt động:

- Trường hợp `lp.algoType == 1` (Thuật toán Bland): Thuật toán duyệt tuần tự từ trái qua phải (chỉ số j tăng dần). Ngay khi bắt gặp cột đầu tiên có chi phí rút gọn mang giá trị âm đáng kể (`tableau[m][j] < -CLEAN_EPS`), hàm lập tức trả về chỉ số cột j đó mà không cần kiểm tra tiếp. Phương pháp lựa chọn chỉ số nhỏ nhất này triệt tiêu hoàn toàn khả năng xảy ra hiện tượng lặp chu kỳ vô hạn trong không gian suy biến.
- Trường hợp mặc định (Thuật toán Dantzig): Hàm khởi tạo một biến lính canh `minVal = -CLEAN_EPS`. Vòng lặp duyệt qua toàn bộ các cột phi cơ sở nhằm tìm ra phần tử có giá trị âm sâu nhất (tức là phần tử nhỏ nhất trên dòng mục tiêu). Việc chọn cột này đảm bảo tốc độ giảm của hàm mục tiêu trên mỗi đơn vị thay

đổi của biến số là lớn nhất, từ đó giảm thiểu tổng số bước lặp trên các bài toán quy mô thực tế.

- Điều kiện tối ưu: Nếu toàn bộ các phần tử trên dòng mục tiêu đều lớn hơn hoặc bằng `-CLEAN_EPS`, hàm trả về `-1`, báo hiệu trạng thái tối ưu toán học đã đạt được.

4.2.6 Tìm kiếm biến ra

Hàm `findPivotRow` áp dụng tiêu chuẩn tỷ số nhỏ nhất để chọn biến cơ sở sẽ bị loại bỏ khỏi cơ sở hiện hành, đảm bảo nghiệm của bảng sau phép biến đổi vẫn nằm trong miền khả thi không âm ($x \geq 0$).

Code thực hiện:

```
1 int SimplexSolver::findPivotRow(int pivotCol) {
2     int m = tableau.size() - 1;
3     int n = tableau[0].size() - 1;
4     int pivotRow = -1;
5     double minRatio = 1e9;
6     int minVarIndex = (int)1e9;
7
8     for (int i = 0; i < m; ++i) {
9         double a_ij = tableau[i][pivotCol];
10        if (a_ij > CLEAN_EPS) {
11            double ratio = tableau[i][n] / a_ij;
12            if (ratio < minRatio - CLEAN_EPS) {
13                minRatio = ratio;
14                pivotRow = i;
15                minVarIndex = basicVariables[i];
16            }
17            else if (std::abs(ratio - minRatio) <= CLEAN_EPS) {
18                if (lp.algoType == 1 && basicVariables[i] <
19                    minVarIndex) {
20                    pivotRow = i;
21                    minVarIndex = basicVariables[i];
22                }
23            }
24        }
25    }
26    return pivotRow;
}
```

Cơ chế hoạt động:

- Sàng lọc hệ số: Thuật toán chỉ xét các hàng có hệ số tại cột xoay mang giá trị dương thực sự lớn hơn không: `if (a_ij > CLEAN_EPS)`. Các hàng có hệ số âm hoặc xấp xỉ không bị bỏ qua vì chúng không đặt ra giới hạn trên cho sự tăng trưởng của biến vào.
- Tỷ số chặn trên: Định nghĩa tỷ số $\theta_i = \frac{\text{tableau}[i][n]}{\text{tableau}[i][\text{pivotCol}]} = \frac{b_i}{a_{ij}}$. Nếu tỷ số mới tìm thấy nhỏ hơn giá trị nhỏ nhất hiện tại một khoảng đáng kể (`ratio < minRatio`

- CLEAN_EPS), hệ thống cập nhật lại minRatio, lưu lại vị trí dòng pivotRow = i, và ghi nhận chỉ số biến cơ sở tương ứng minVarIndex.
- Xử lý trường hợp suy biến: Khi xảy ra tình trạng bằng nhau về tỷ số trong phạm vi sai số thực (`std::abs(ratio - minRatio) <= CLEAN_EPS`), nếu giải thuật đang chạy dưới cấu hình Bland (`lp.algoType == 1`), hệ thống sẽ ưu tiên chọn dòng nào đang giữ biến cơ sở có chỉ số định danh nhỏ hơn (`basicVariables[i] < minVarIndex`). Sự phối hợp chặt chẽ giữa việc chọn cột nhỏ nhất ở `findPivotColumn` và chọn dòng có biến cơ sở nhỏ nhất ở đây chính là cốt lõi của Định lý Bland.
- Bài toán không giới nội: Nếu kết thúc vòng lặp mà pivotRow vẫn giữ nguyên giá trị khởi tạo -1, điều đó chứng tỏ biến vào có thể tăng lên vô hạn mà không vi phạm bất kỳ ràng buộc nào, tức là bài toán không giới nội.

4.2.7 Phép biến đổi khử toàn phần Gauss-Jordan

Hàm `performPivot` thực thi phép biến đổi đại số tuyến tính trên ma trận nhằm chuyển đổi cột xoay `pivotCol` thành một vector đơn vị mới $e_{pivotRow}$, qua đó cập nhật nghiệm cơ sở mới.

Code thực hiện:

```
1 void SimplexSolver::performPivot(int pivotRow, int pivotCol) {
2     int m = tableau.size();
3     int n = tableau[0].size();
4     double pivotValue = tableau[pivotRow][pivotCol];
5
6     for (int j = 0; j < n; ++j) {
7         tableau[pivotRow][j] /= pivotValue;
8         if (std::abs(tableau[pivotRow][j]) < CLEAN_EPS)
9             tableau[pivotRow][j] = 0.0;
10    }
11    for (int i = 0; i < m; ++i) {
12        if (i != pivotRow) {
13            double factor = tableau[i][pivotCol];
14            for (int j = 0; j < n; ++j) {
15                tableau[i][j] -= factor * tableau[pivotRow][j];
16                if (std::abs(tableau[i][j]) < CLEAN_EPS)
17                    tableau[i][j] = 0.0;
18            }
19        }
20    }
21    basicVariables[pivotRow] = pivotCol;
22    iterationCount++;
23    recordStep(QString("Tu vung %1").arg(iterationCount + 1));
24 }
```

Cơ chế hoạt động:

- Chuẩn hóa hàng xoay: Toàn bộ các phần tử thuộc hàng `pivotRow` được chia cho phần tử trục `pivotValue`. Thao tác này biến đổi phần tử tại vị trí `tableau[pivotRow][pivotCol]` thành chính xác 1.0. Để triệt tiêu sai số số thực sau phép chia, bộ lọc tạp nhiễu

được áp dụng: nếu giá trị sau tính toán nhỏ hơn `CLEAN_EPS`, nó lập tức bị cưỡng bức về 0.0.

- Khử toàn bộ hàng khác: Với mỗi hàng $i \neq \text{pivotRow}$, ta thực hiện phép nhân dòng `pivotRow` mới với hệ số tỷ lệ `factor = tableau[i][pivotCol]` rồi trừ trực tiếp vào dòng i :

$$\text{tableau}[i][j] \leftarrow \text{tableau}[i][j] - \text{factor} \times \text{tableau}[\text{pivotRow}][j]$$

Phép toán này triệt tiêu tất cả các hệ số trên cột xoay ngoại trừ dòng xoay về giá trị 0.0. Hệ thống tiếp tục lọc sai số số thực trên từng cell của bảng đơn hình để duy trì tính ổn định số học.

- Cập nhật cấu trúc điều khiển: Đổi định danh biến cơ sở quản lý hàng `pivotRow` sang chỉ số cột mới chèn vào: `basicVariables[pivotRow] = pivotCol`. Tăng số bước lặp và lưu trữ bảng đơn hình mới vào lịch sử để giao diện người dùng có thể hiển thị từng bước tiến triển của thuật toán.

4.2.8 Vòng lặp thực thi tiến trình

Hàm `runSimplexLoop` đóng vai trò điều phối vòng lặp, kiểm soát điều kiện dừng tối ưu, phát hiện lỗi lặp vô hạn và bài toán không giới nội.

Code thực hiện:

```
1 bool SimplexSolver::runSimplexLoop(bool isPhaseOne) {
2     const int MAX_ITERATIONS = 500;
3     while (true) {
4         if (iterationCount > MAX_ITERATIONS) {
5             statusMsg = "Hien tuong xoay vong (Cycling)!\nThuat
6                 toan bi lap vo han.";
7             return false;
8         }
9         int pivotCol = findPivotColumn();
10        if (pivotCol == -1) return true;
11        int pivotRow = findPivotRow(pivotCol);
12        if (pivotRow == -1) {
13            statusMsg = "Bai toan khong gioi noi";
14            if (!history.empty()) {
15                history.back().isUnbounded = true;
16            }
17            return false;
18        }
19        if (!history.empty()) {
20            history.back().pivotRow = pivotRow;
21            history.back().pivotCol = pivotCol;
22        }
23        performPivot(pivotRow, pivotCol);
24    }
```

Cơ chế hoạt động:

- Cơ chế bảo vệ lặp vòng: Đặt ra một hằng số cứng $MAX_ITERATIONS = 500$. Nếu thuật toán rơi vào trạng thái suy biến trầm trọng và không sử dụng quy tắc Bland, nó có thể xoay quanh các đỉnh của đa diện mà không bao giờ thoát ra được. Khi `iterationCount` vượt quá 500, hàm lập tức ngắt tiến trình, ghi nhận thông điệp lỗi hệ thống và trả về `false`.
- Thực thi các bước tuần tự: Trong mỗi vòng lặp vô hạn `while (true)`:
 - + Xác định cột xoay thông qua `findPivotColumn()`. Nếu kết quả là -1, hệ thống đã đạt trạng thái tối ưu toàn cục, hàm trả về `true` để thoát vòng lặp thành công.
 - + Xác định dòng xoay thông qua `findPivotRow(pivotCol)`. Nếu kết quả trả về là -1, hàm xác nhận bài toán không giới nội, cập nhật cờ hiệu trạng thái `isUnbounded = true` vào bước lịch sử cuối cùng và trả về `false`.
 - + Nếu cả dòng và cột xoay đều hợp lệ, thông tin tọa độ xoay được cập nhật vào bản ghi lịch sử phục vụ hiển thị đồ họa, sau đó lệnh `performPivot(pivotRow, pivotCol)` được thực thi để chuyển sang bảng tiếp theo.

4.2.9 Giải thuật toán Đơn hình Hai pha

Hàm `solveTwoPhase` chịu trách nhiệm giải quyết trường hợp bài toán có vector về phải chứa các phần tử âm (nghiệm xuất phát ban đầu không khả thi).

Code thực hiện:

```
1 bool SimplexSolver::solveTwoPhase() {
2     int m = lp.A.size();
3     int n_slack = lp.c.size();
4
5     buildTableau();
6     if (!history.empty()) history.pop_back();
7     int cols = tableau[0].size() - 1;
8
9     for (int i = 0; i <= m; ++i) {
10         tableau[i].insert(tableau[i].begin() + cols, 0.0);
11     }
12     int a_col = cols;
13     cols++;
14
15     for (int i = 0; i < m; ++i) {
16         tableau[i][a_col] = -1.0;
17     }
18
19     for (int j = 0; j <= cols; ++j) tableau[m][j] = 0.0;
20     tableau[m][a_col] = 1.0;
21     recordStep("Tu vung 1 (Khoi tao Pha 1)");
22
23     int pivotRow = -1;
24     double minRhs = 0.0;
25     for (int i = 0; i < m; ++i) {
```

```
26         if (tableau[i][cols] < minRhs - CLEAN_EPS) {
27             minRhs = tableau[i][cols];
28             pivotRow = i;
29         }
30     }
31
32     if (pivotRow != -1) {
33         if (!history.empty()) {
34             history.back().pivotRow = pivotRow;
35             history.back().pivotCol = a_col;
36         }
37         performPivot(pivotRow, a_col);
38     }
39
40     if (!this->runSimplexLoop()) return false;
41
42     if (std::abs(tableau[m][cols]) > CLEAN_EPS) {
43         statusMsg = "Vo nghiem (Min epsilon > 0)";
44         if (!history.empty()) {
45             history.back().isInfeasible = true;
46         }
47         return false;
48     }
49
50     iterationCount = 0;
51     for (int j = 0; j < n_slack; ++j) {
52         tableau[m][j] = lp.c[j];
53     }
54     tableau[m][a_col] = 1e9;
55     tableau[m][cols] = lp.c_0;
56     for (int i = 0; i < m; ++i) {
57         int b_col = basicVariables[i];
58         double factor = tableau[m][b_col];
59         if (std::abs(factor) > CLEAN_EPS) {
60             for (int j = 0; j <= cols; ++j) {
61                 tableau[m][j] -= factor * tableau[i][j];
62             }
63         }
64     }
65
66     this->recordStep("Tu vung 1 (Khoi tao Pha 2)");
67     bool success = this->runSimplexLoop();
68
69     if (success) {
70         if (this->checkAlternativeOptima()) {
71             this->firstSolution = this->getSolution();
72             findAndRecordAlternativeOptimum();
73
74             bool isReallyDifferent = false;
75             for (size_t i = 0; i < firstSolution.size(); ++i) {
```

```

76         if (std::abs(firstSolution[i] - altSolution[i])
77             > CLEAN_EPS) {
78             isReallyDifferent = true;
79             break;
80         }
81
82         if (isReallyDifferent) {
83             statusMsg = "Vo so nghiem";
84         } else {
85             statusMsg = "Toi uu";
86             if (!history.empty() && history.back().stepName
87                 == "Diem toi uu thu 2") {
88                 history.pop_back();
89             }
90         } else {
91             statusMsg = "Toi uu";
92             this->firstSolution = this->getSolution();
93             this->altSolution = this->firstSolution;
94         }
95     }
96     return success;
97 }

```

Cơ chế hoạt động:

- Thiết lập cấu trúc Pha 1: Bản chất của Pha 1 là đưa một biến nhân tạo x_0 (`a_col`) vào tất cả các dòng với hệ số -1.0 . Ma trận `tableau` được mở rộng cột bằng lệnh `insert`. Dòng mục tiêu (dòng m) được làm sạch về 0.0 , ngoại trừ vị trí cột nhân tạo được gán bằng 1.0 . Mục tiêu toán học lúc này là tối thiểu hóa giá trị của biến nhân tạo này.
- Khởi động bước xoay đầu tiên: Tìm dòng có giá trị về phải âm sâu nhất (`minRhs`). Ép cột nhân tạo `a_col` xoay với dòng này bằng lệnh `performPivot(pivotRow, a_col)`. Phép xoay đặc biệt này lập tức đưa toàn bộ các giá trị âm ở cột về phải trở thành số dương, thiết lập trạng thái khả thi cho bảng Pha 1.
- Giải Pha 1: Gọi `runSimplexLoop()` để tối ưu hóa hàm mục tiêu Pha 1. Khi dừng, kiểm tra giá trị tối ưu tại ô nghiệm `tableau[m][cols]`. Nếu giá trị này lớn hơn `CLEAN_EPS`, điều đó chứng tỏ biến nhân tạo không thể triệt tiêu về 0 , đồng nghĩa hệ ràng buộc gốc xung đột hoàn toàn $\rightarrow$ Bài toán vô nghiệm (`isInfeasible = true`).
- Cấu trúc Pha 2: Nếu biến nhân tạo triệt tiêu về 0 , hệ thống chuyển sang Pha 2 bằng cách nạp lại hệ số chi phí thực tế từ `lp.c` vào dòng mục tiêu. Do các biến cơ sở hiện hành có thể có hệ số khác không trên dòng mục tiêu mới nạp, thuật toán tiến hành vòng lặp khử Gauss (`tableau[m][j] -= factor * tableau[i][j]`) để đưa dòng kiểm tra về đúng cấu trúc chuẩn của từ vựng hiện tại.

- Giải Pha 2 và phân tích cấu trúc đa nghiệm: Tiến hành gọi `runSimplexLoop()` lần hai để tìm nghiệm tối ưu thực tế. Nếu tối ưu thành công, hàm kích hoạt kiểm tra tính đa nghiệm thông qua `checkAlternativeOptima()`. Nếu phát hiện có khả năng đa nghiệm, hệ thống gọi tiếp `findAndRecordAlternativeOptimum()` để tìm điểm cực biên thứ hai, rồi thực hiện phép so sánh khoảng cách thực để kết luận bài toán có vô số nghiệm thực sự hay chỉ là nghiệm ảo trùng lặp.

4.2.10 Rà soát khả năng đa nghiệm

Hàm `checkAlternativeOptima` chịu trách nhiệm phân tích trạng thái hình học của bài toán tại điểm tối ưu hiện hành nhằm phát hiện dấu hiệu tồn tại của tập nghiệm vô số.

Code thực hiện:

```
1 bool SimplexSolver::checkAlternativeOptima() {
2     int m = tableau.size() - 1;
3     int n = tableau[0].size() - 1;
4
5     for (int j = 0; j < n; ++j) {
6         if (std::abs(tableau[m][j]) >= 1e8) continue;
7
8         bool isBasic = false;
9         for (int i = 0; i < m; ++i) {
10             if (basicVariables[i] == j) {
11                 isBasic = true;
12                 break;
13             }
14         }
15
16         if (!isBasic && std::abs(tableau[m][j]) <= CLEAN_EPS) {
17             return true;
18         }
19     }
20
21     return false;
22 }
```

Cơ chế hoạt động:

- **Bộ lọc cấu trúc Big-M:** Vòng lặp duyệt qua tất cả các cột j của bảng đơn hình. Lệnh `if (std::abs(tableau[m][j]) >= 1e8) continue;` đóng vai trò cô lập biến giả nhân tạo x_0 của Pha 1 đã bị ép phạt bằng Big-M trong Pha 2, ngăn chặn việc nhận diện sai lệch các biến hình thức này thành biến tối ưu kề cận.
- **Xác định trạng thái phi cơ sở:** Thuật toán duyệt qua mảng điều khiển cơ sở `basicVariables` để đảm bảo cột j đang xét không thuộc hệ biến cơ sở hiện hành (`!isBasic`).
- **Đánh giá chỉ phí rút gọn tối ưu:** Nếu biến phi cơ sở j thỏa mãn điều kiện có hệ số hàm mục tiêu xấp xỉ không (`std::abs(tableau[m][j]) <= CLEAN_EPS`), điều này chứng tỏ hình học đa diện tại đỉnh này có ít nhất một cạnh đi tới một

đỉnh kề có cùng giá trị hàm mục tiêu. Hàm lập tức trả về `true`, kích hoạt tiến trình tìm kiếm điểm cực biên thứ hai.

4.2.11 Tìm kiếm và xác thực điểm cực biên thứ hai

Hàm `findAndRecordAlternativeOptimum` hiện thực hóa kỹ thuật quay thử trên các cột phi cơ sở tiềm năng nhằm mục đích kiến thiết và kiểm tra khoảng cách thực tế giữa hai điểm cực biên tối ưu.

Code thực hiện:

```
1 void SimplexSolver::findAndRecordAlternativeOptimum() {
2     int m = tableau.size() - 1;
3     int n = tableau[0].size() - 1;
4
5     auto backupTableau = this->tableau;
6     auto backupBasicVars = this->basicVariables;
7     auto backupHistory = this->history;
8
9     bool foundDifferent = false;
10
11     for (int j = 0; j < n; ++j) {
12         if (std::abs(tableau[m][j]) >= 1e8) continue;
13
14         bool isBasic = false;
15         for (int i = 0; i < m; ++i) {
16             if (basicVariables[i] == j) {
17                 isBasic = true;
18                 break;
19             }
20         }
21
22         if (!isBasic && std::abs(tableau[m][j]) <= CLEAN_EPS) {
23             int altPivotRow = findPivotRow(j);
24
25             if (altPivotRow == -1) {
26                 continue;
27             }
28
29             if (!history.empty()) {
30                 history.back().pivotRow = altPivotRow;
31                 history.back().pivotCol = j;
32             }
33
34             performPivot(altPivotRow, j);
35             std::vector<double> tempSol = this->getSolution();
36
37             bool isDifferent = false;
38             for (size_t k = 0; k < firstSolution.size(); ++k) {
39                 if (std::abs(firstSolution[k] - tempSol[k]) >
40                     CLEAN_EPS) {
```

```
40         isDifferent = true;
41         break;
42     }
43 }
44
45 if (isDifferent) {
46     this->altSolution = tempSol;
47     if (!history.empty()) {
48         history.back().stepName = "Diem toi uu thu
49             2";
50     }
51     foundDifferent = true;
52
53     this->tableau = backupTableau;
54     this->basicVariables = backupBasicVars;
55     break;
56 } else {
57     this->tableau = backupTableau;
58     this->basicVariables = backupBasicVars;
59     this->history = backupHistory;
60 }
61 }
62
63 if (!foundDifferent) {
64     this->altSolution = this->firstSolution;
65 }
66 }
```

Cơ chế hoạt động:

1. **Thiết lập vùng đệm phục hồi:** Trước khi can thiệp sâu vào cấu trúc đại số, hàm thực hiện sao lưu sâu ma trận bảng đơn hình, mảng biến cơ sở và cấu trúc lịch sử. Điều này đặc biệt quan trọng trong các bài toán quy hoạch tuyến tính suy biến, nơi nhiều tập cơ sở khác nhau cùng đại diện cho một đỉnh hình học duy nhất.
2. **Lựa chọn và kiểm tra điều kiện dòng trục xoay:** Khi phát hiện cột tiềm năng j có reduced cost bằng 0, thuật toán gọi `findPivotRow(j)`. Nếu hàm trả về -1 (tức là hướng di chuyển của cạnh là vô hạn trong miền khả thi nhưng chi phí không đổi), hệ thống không thực hiện xoay trên cột này mà dùng lệnh `continue` để chuyển sang kiểm tra các cột ứng viên tiếp theo.
3. **Ánh xạ ngược và xác thực khoảng cách Euclid:** Sau khi lệnh `performPivot` dịch chuyển từ vùng sang trạng thái cơ sở mới, hàm kích hoạt `getSolution()`. Hàm này sẽ giải mã các biến nội bộ (bao gồm việc tính hiệu $x_j^+ - x_j^-$ của các biến tự do ban đầu) để trả về mảng nghiệm nguyên bản của đề bài. Vòng lặp kiểm tra sai số tuyệt đối `if (std::abs(firstSolution[k] - tempSol[k]) > CLEAN_EPS)` đảm bảo rằng sự thay đổi về ma trận cơ sở nội bộ thực sự tạo ra một điểm cực biên hình học mới trong không gian nghiệm thực tế.

4. Xử lý kết quả hậu kiểm:

- *Nếu nghiệm thực sự khác biệt:* Ghi nhận điểm cực biên tối ưu thứ hai vào `altSolution`, đánh dấu bước lật trang lịch sử đồ họa là "Điểm tối ưu thứ 2", khôi phục lại bảng đơn hình gốc để luồng logic chính của chương trình tiếp tục vận hành không bị sai lệch cấu trúc dữ liệu.
- *Nếu nghiệm trùng nhau (Nghiệm ảo do suy biến hình học):* Hệ thống tiến hành rollback trạng thái ma trận hoàn toàn về nguyên trạng ban đầu thông qua các biến sao lưu, sau đó tiếp tục tìm kiếm trên các cột phi cơ sở còn lại. Nếu duyệt hết toàn bộ ma trận mà cờ `foundDifferent` vẫn mang giá trị `false`, thuật toán đồng nhất `altSolution` bằng `firstSolution`, báo hiệu không tồn tại điểm tối ưu biên độc lập thứ hai.

4.2.12 Trích xuất và tái cấu trúc nghiệm thực tế

Hàm `getSolution` chuyển đổi các giá trị thô trên bảng đơn hình mở rộng về đúng dạng không gian vector ban đầu của bài toán do người dùng định nghĩa.

Code thực hiện:

```
1 std::vector<double> SimplexSolver::getSolution() const {
2     int origCount = (int)originalVarBounds.size();
3     std::vector<double> sol(origCount, 0.0);
4     if (this->tableau.empty()) return sol;
5     int m = this->tableau.size() - 1;
6     int n = this->tableau[0].size() - 1;
7
8     std::vector<double> extendedSol(this->tableau[0].size() -
9         1, 0.0);
10    for (int i = 0; i < m; i++) {
11        int basicVar = this->basicVariables[i];
12        if (basicVar >= 0 && basicVar < (int)extendedSol.size()
13            )
14            extendedSol[basicVar] = this->tableau[i][n];
15    }
16
17    int internalIdx = 0;
18    for (int i = 0; i < origCount; ++i) {
19        const VarBound& vb = originalVarBounds[i];
20        if (vb.isFree || vb.sign == "free") {
21            sol[i] = extendedSol[internalIdx] - extendedSol[
22                internalIdx + 1];
23            internalIdx += 2;
24        } else if (vb.sign == "<=") {
25            sol[i] = -extendedSol[internalIdx];
26            internalIdx += 1;
27        } else {
28            sol[i] = extendedSol[internalIdx];
29            internalIdx += 1;
30        }
31    }
32 }
```

```
29     return sol;  
30 }
```

Cơ chế hoạt động:

- Thu thập giá trị biến mở rộng: Hàm khởi tạo vector `extendedSol` đại diện cho giá trị của tất cả các biến số trong bảng đơn hình chuẩn hóa (bao gồm cả biến bù và biến nhân tạo). Duyệt qua mảng `basicVariables`, nếu một dòng i do biến cơ sở `basicVar` quản lý, giá trị nghiệm của nó chính là phần tử về phải nằm ở cột cuối cùng: `extendedSol[basicVar] = this->tableau[i][n]`. Các biến không thuộc cơ sở giữ nguyên giá trị khởi tạo 0.0.
- Ánh xạ ngược về miền nghiệm gốc: Tiến hành duyệt qua danh sách cấu hình biên nguyên bản `originalVarBounds` để tái cấu trúc vector kết quả `sol`:
 - + Nếu gốc là biến tự do: Giá trị thực tế bằng hiệu của hai biến mở rộng liên tiếp: `sol[i] = extendedSol[internalIdx] - extendedSol[internalIdx + 1]`. Bộ đếm chỉ số bảng tăng thêm 2 đơn vị.
 - + Nếu gốc là biến không dương $x_i \leq 0$: Giá trị thực tế bằng dấu âm của biến mở rộng tương ứng: `sol[i] = -extendedSol[internalIdx]`. Bộ đếm chỉ số tăng 1.
 - + Trường hợp chuẩn ($x_i \geq 0$): Giá trị thực tế được sao chép nguyên vẹn từ `extendedSol[internalIdx]`.

4.3 Độ phức tạp thuật toán

Giả sử m là số lượng ràng buộc và n là tổng số biến sau khi chuẩn hóa.

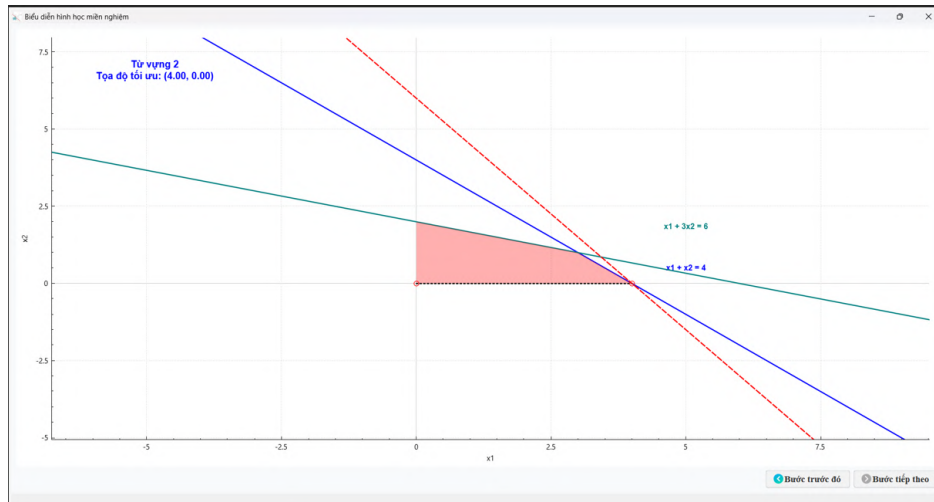
4.3.1 Độ phức tạp không gian (Space Complexity)

- **Bảng đơn hình (tableau):** Ma trận double kích thước $(m+1) \times (n+1)$ chiếm khoảng $\mathcal{O}(m \times n)$.
- **Hệ thống lưu vết (history):** Lưu lại bảng ở mỗi bước lặp phục vụ đồ họa. Nếu có I bước lặp, chi phí là $\mathcal{O}(I \times m \times n)$.
- **Tổng kết:** Do I bị giới hạn tĩnh (`MAX_ITERATIONS = 500`), không gian tổng thể bị chặn ở mức $\mathcal{O}(m \times n)$. Với bài toán cỡ lớn trong môn học ($m, n < 200$), chỉ tiêu tốn vài MB RAM, tuyệt đối an toàn không lo tràn bộ nhớ (Memory Overflow).

4.3.2 Độ phức tạp thời gian (Time Complexity)

- **Chi phí một vòng lặp:** Bị chi phối hoàn toàn bởi hàm `performPivot` (Khử Gauss-Jordan), đòi hỏi duyệt qua toàn bộ ma trận. Số phép tính (FLOPs) là $\mathcal{O}(m \times n)$.
- **Trường hợp xấu nhất (Worst-Case):** Theo Klee-Minty, thuật toán đơn hình cơ bản có thể phải đi qua mọi đỉnh của đa diện, dẫn đến độ phức tạp thời gian hàm mũ $\mathcal{O}(2^m \times m \times n)$.

- **Phân tích thực nghiệm (Smoothed Analysis):** Lý thuyết phân tích nhẵn của Spielman và Teng chỉ ra rằng nhờ cấu trúc nhiễu của số liệu thực tế, số bước lặp I trung bình luôn tỷ lệ tuyến tính với số ràng buộc $\mathcal{O}(m)$.
- **Tổng kết:** Thời gian thực thi thực tế đạt xấp xỉ $\mathcal{O}(m^2 \times n)$. Giải thuật cung cấp tốc độ xử lý tính bằng mili-giây, cực kỳ mượt mà.



Hình 5.1.4: Biểu diễn hình học của bài toán bằng phần mềm

Nhận xét: Dựa vào cả 2 kết quả giải bằng tay và giải bằng phần mềm đều cho cùng 1 từ vựng và kết quả $\max 3x_1 + 2x_2 = 12$ và biểu diễn hình học của cả hai cách giải tay và giải bằng phần mềm đều giống nhau.

5.2 Thử nghiệm với bài toán vô nghiệm

Ta xét bài toán quy hoạch tuyến tính sau:

$$\begin{aligned} \max \quad & 3x_1 + 2x_2 \\ & 2x_1 + x_2 \leq 2 \\ & 3x_1 + 4x_2 \geq 12 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Ta giải bài toán quy hoạch này bằng tay và bằng phần mềm ta được kết quả sau:

Pha 1:

Từ vựng 1 (Xuất phát):

$$\begin{cases} \varepsilon = \\ w_1 = 2 - 2x_1 - x_2 + x_0 \\ \leftarrow w_2 = -12 + 3x_1 + 4x_2 + x_0 \end{cases}$$

Hình 5.2.1: So sánh giữa giải tay và phần mềm

Từ vựng 2:

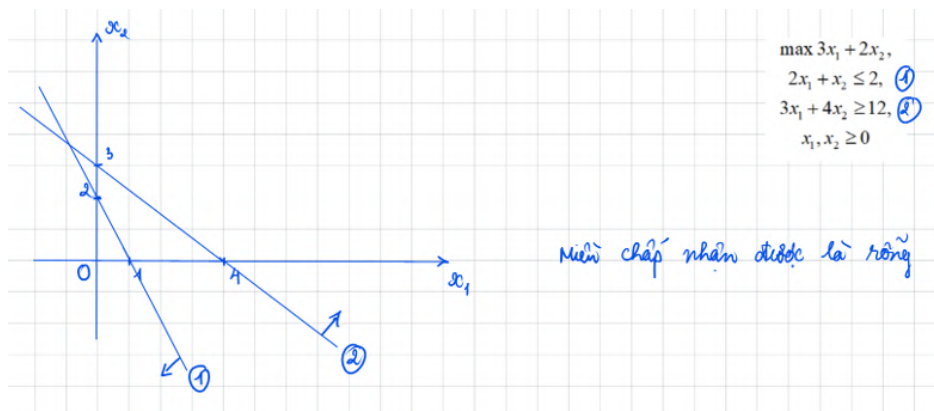
$\begin{cases} \varepsilon = 12 - 3x_1 - 4x_2 + w_2 \\ w_1 = 14 - 5x_1 - 5x_2 + w_2 \\ x_0 = 12 - 3x_1 - 4x_2 + w_2 \end{cases}$	$\begin{aligned} \xi &= 12.00 - 3.00 x_1 - 4.00 x_2 + 1.00 w_2 \\ w_1 &= 14.00 - 5.00 x_1 - 5.00 x_2 + 1.00 w_2 \\ x_0 &= 12.00 - 3.00 x_1 - 4.00 x_2 + 1.00 w_2 \end{aligned}$
--	---

Hình 5.2.2: So sánh giữa giải tay và phần mềm

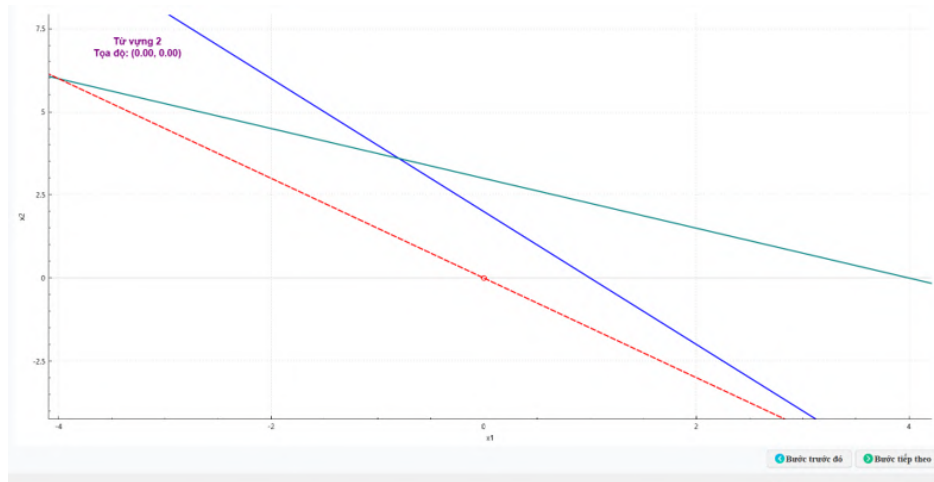
Từ vựng 3 (tối ưu):

$\begin{cases} \varepsilon = 4/5 + x_1 + 4/5 w_1 + 1/5 w_2 \\ x_0 = 4/5 + x_1 + 4/5 w_1 + 1/5 w_2 \\ x_2 = 4/5 - x_1 - 1/5 w_1 + 1/5 w_2 \end{cases}$	$\begin{aligned} \xi &= 0.80 + 1.00 x_1 + 0.80 w_1 + 0.20 w_2 \\ x_2 &= 2.80 - 1.00 x_1 - 0.20 w_1 + 0.20 w_2 \\ x_0 &= 0.80 + 1.00 x_1 + 0.80 w_1 + 0.20 w_2 \end{aligned}$
---	--

Hình 5.2.3: So sánh giữa giải tay và phần mềm



Hình 5.2.4: Biểu diễn hình học của bài toán khi giải tay



Hình 5.2.5: Biểu diễn hình học của bài toán bằng phần mềm

Nhận xét: Dựa vào cả 2 kết quả giải bằng tay và giải bằng phần mềm đều cho cùng 1 kết quả là vô nghiệm và biểu diễn hình học của cả hai cách giải tay và giải bằng phần mềm đều giống nhau.

5.3 Thử nghiệm với bài toán không giới nội

Xét bài toán quy hoạch tuyến tính sau:

$$\begin{aligned} \max \quad & x_1 - x_2 \\ & -2x_1 + x_2 \leq -1 \\ & -x_1 - 2x_2 \leq -2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Ta giải bài toán quy hoạch này bằng tay và bằng phần mềm ta được kết quả sau:

Pha 1:

Từ vựng 1 (xuất phát):

$\begin{aligned} \xi = & \\ \left\{ \begin{aligned} w_1 &= -1 + 2x_1 - x_2 + x_0 \\ w_2 &= -2 + x_1 + 2x_2 + x_0 \end{aligned} \right. \end{aligned}$	$\xi =$	1.00	x0
	w1 =	-1.00	+ 2.00 x1 - 1.00 x2 + 1.00 x0
	w2 =	-2.00	+ 1.00 x1 + 2.00 x2 + 1.00 x0

Hình 5.3.1: So sánh giữa giải tay và phần mềm

Từ vựng 2:

$\begin{cases} \varepsilon = 2 - x_1 - 2x_2 + w_2 \\ w_1 = 1 + x_1 - 3x_2 + w_2 \\ x_0 = 2 - x_1 - 2x_2 + w_2 \end{cases}$	$\begin{aligned} \xi &= 2.00 - 1.00 x_1 - 2.00 x_2 + 1.00 w_2 \\ w_1 &= 1.00 + 1.00 x_1 - 3.00 x_2 + 1.00 w_2 \\ x_0 &= 2.00 - 1.00 x_1 - 2.00 x_2 + 1.00 w_2 \end{aligned}$
--	--

Hình 5.3.2: So sánh giữa giải tay và phần mềm

Từ vệtng 3:

$\begin{cases} \varepsilon = \frac{4}{3} - \frac{5}{3}x_1 + \frac{1}{3}w_2 + \frac{2}{3}w_1 \\ x_0 = \frac{4}{3} - \frac{5}{3}x_1 + \frac{1}{3}w_2 + \frac{2}{3}w_1 \\ x_2 = \frac{1}{3} + \frac{1}{3}x_1 + \frac{1}{3}w_2 - \frac{1}{3}w_1 \end{cases}$	$\begin{aligned} \xi &= 1.33 - 1.67 x_1 + 0.67 w_1 + 0.33 w_2 \\ x_2 &= 0.33 + 0.33 x_1 - 0.33 w_1 + 0.33 w_2 \\ x_0 &= 1.33 - 1.67 x_1 + 0.67 w_1 + 0.33 w_2 \end{aligned}$
--	--

Hình 5.3.3: So sánh giữa giải tay và phần mềm

Từ vệtng 4 (tối ưu pha 1):

$\begin{cases} \varepsilon = \frac{4}{3} - \frac{5}{3}x_1 + \frac{1}{3}w_2 + \frac{2}{3}w_1 \\ x_0 = \frac{4}{3} - \frac{5}{3}x_1 + \frac{1}{3}w_2 + \frac{2}{3}w_1 \\ x_2 = \frac{1}{3} + \frac{1}{3}x_1 + \frac{1}{3}w_2 - \frac{1}{3}w_1 \end{cases}$	$\begin{aligned} \xi &= 1.33 - 1.67 x_1 + 0.67 w_1 + 0.33 w_2 \\ x_2 &= 0.33 + 0.33 x_1 - 0.33 w_1 + 0.33 w_2 \\ x_0 &= 1.33 - 1.67 x_1 + 0.67 w_1 + 0.33 w_2 \end{aligned}$
--	--

Hình 5.3.4: So sánh giữa giải tay và phần mềm

Pha 2:
Từ vệt 1:

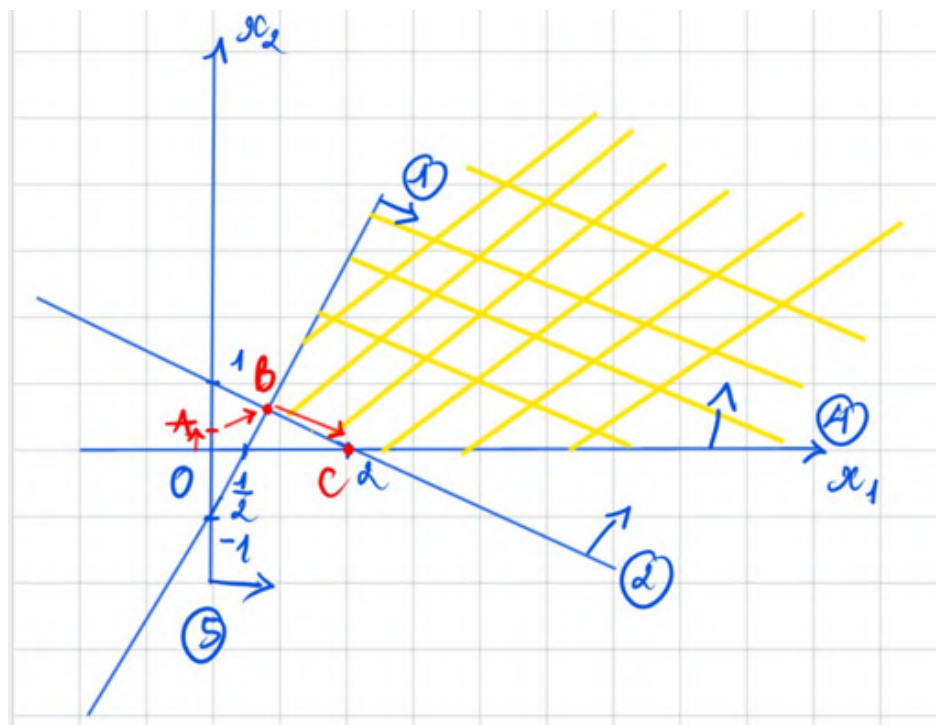
$\begin{cases} Z = -1/5 + 1/5 w_2 - 3/5 w_1 \\ x_1 = 3/5 + 2/5 w_2 - 1/5 w_1 \\ x_2 = 4/5 + 1/5 w_2 + 2/5 w_1 \end{cases}$	$\begin{aligned} -Z &= -0.20 - 0.60 w_1 + 0.20 w_2 \\ x_2 &= 0.60 - 0.20 w_1 + 0.40 w_2 \\ x_1 &= 0.80 + 0.40 w_1 + 0.20 w_2 \end{aligned}$
--	---

Hình 5.3.5: So sánh giữa giải tay và phần mềm

Từ vệt 2 (tối ưu pha 2):

$\begin{cases} Z = -2 - w_2 + 3x_1 \\ x_2 = 2 + w_2 - 2x_1 \\ w_1 = 3 + 2w_2 - 5x_1 \end{cases}$	$\begin{aligned} -Z &= -2.00 + 3.00 x_2 - 1.00 w_2 \\ w_1 &= 3.00 - 5.00 x_2 + 2.00 w_2 \\ x_1 &= 2.00 - 2.00 x_2 + 1.00 w_2 \end{aligned}$
--	---

Hình 5.3.6: So sánh giữa giải tay và phần mềm



Hình 5.3.7: Biểu diễn hình học của bài toán khi giải tay



Hình 5.3.8: Biểu diễn hình học của bài toán bằng phần mềm

Nhận xét: Dựa vào cả 2 kết quả giải bằng tay và giải bằng phần mềm đều cho kết quả các từ vịnh giống nhau và cùng 1 kết quả là bài toán không giới nội và biểu diễn hình học của cả hai cách giải tay và giải bằng phần mềm đều giống nhau.

5.4 Thử nghiệm với bài toán vô số nghiệm

Ta xét bài toán quy hoạch tuyến tính sau:

$$\begin{aligned} \max \quad & x_1 + x_2 \\ & x_1 + x_2 \leq 4 \\ & x_1 \leq 3 \\ & x_2 \leq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Ta giải bài toán quy hoạch này bằng tay và bằng phần mềm ta được kết quả sau:
Từ vịnh 1 (xuất phát):

$\begin{aligned} Z &= -x_1 - x_2 \\ w_1 &= 4 - x_1 - x_2 \\ w_2 &= 3 - x_1 \\ w_3 &= 3 - x_2 \end{aligned}$	$\begin{aligned} -Z &= 0.00 - 1.00 x_1 - 1.00 x_2 \\ w_1 &= 4.00 - 1.00 x_1 - 1.00 x_2 \\ w_2 &= 3.00 - 1.00 x_1 \\ w_3 &= 3.00 - 1.00 x_2 \end{aligned}$
---	---

Hình 5.4.1: So sánh giữa giải tay và phần mềm

Từ vựng 2:

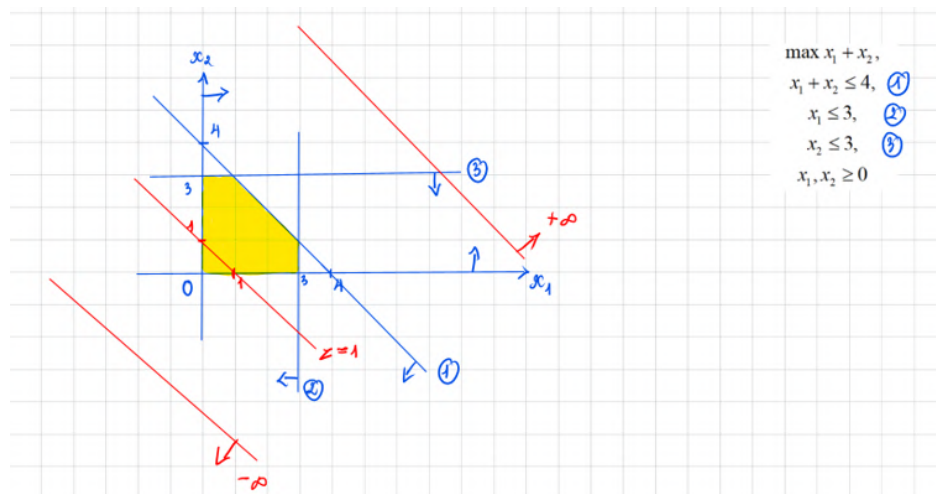
$\begin{cases} z = -3 - x_2 + w_2 \\ w_1 = 1 - x_2 + w_2 \\ w_3 = 3 - w_2 \\ x_1 = 3 - w_2 \end{cases}$	$\begin{aligned} -Z &= -3.00 - 1.00 x_2 + 1.00 w_2 \\ \hline w_1 &= 1.00 - 1.00 x_2 + 1.00 w_2 \\ x_1 &= 3.00 - 1.00 w_2 \end{aligned}$
---	---

Hình 5.4.2: So sánh giữa giải tay và phần mềm

Từ vựng 3:

$\begin{cases} z = -4 + w_1 \\ w_3 = 3 - w_2 \\ x_1 = 3 - w_2 \\ x_2 = 1 + w_2 - w_1 \end{cases}$	$\begin{aligned} -Z &= -4.00 + 1.00 w_1 \\ \hline x_2 &= 1.00 - 1.00 w_1 + 1.00 w_2 \\ x_1 &= 3.00 - 1.00 w_2 \\ w_3 &= 2.00 + 1.00 w_1 - 1.00 w_2 \end{aligned}$
---	---

Hình 5.4.3: So sánh giữa giải tay và phần mềm



Hình 5.4.4: Biểu diễn hình học của bài toán bằng tay



Hình 5.4.5: Biểu diễn hình học của bài toán bằng phần mềm

Nhận xét: Dựa vào cả 2 kết quả giải bằng tay và giải bằng phần mềm đều cho kết quả các từ vựng giống nhau và cùng 1 kết quả là bài toán có vô số nghiệm và $\max x_1 + x_2 = 4$ và biểu diễn hình học của cả hai cách giải tay và giải bằng phần mềm đều giống nhau.

5.5 Thử nghiệm với bài toán xoay vòng

Ta xét bài toán quy hoạch tuyến tính xoay vòng sau:

$$\begin{aligned} \min \quad & -10x_1 + 57x_2 + 9x_3 + 24x_4 \\ & 0.5x_1 - 5.5x_2 - 2.5x_3 + 9x_4 \leq 0 \\ & 0.5x_1 - 1.5x_2 - 0.5x_3 + x_4 \leq 0 \\ & x_1 \leq 1 \\ & x_1, x_2, x_3, x_4 \geq 0 \end{aligned}$$

Số biến: 4 OK

Số ràng buộc: 3

Nhập vào các hệ số của hàm mục tiêu:

	x0	x1	x2	x3	x4	
	0.00	-10.00	57.00	9.00	24.00	Min

Nhập vào các ràng buộc:

	x1	x2	x3	x4	Sign	b_i	
1	0.50	-5.50	-2.50	9.00	<=	0.00	
2	0.50	-1.50	-0.50	1.00	<=	0.00	
3	1.00	0.00	0.00	0.00	<=	1.00	

Reset

Nhập vào các ràng buộc cho biến:

	x1	x2	x3	x4	
1	x1	>=			0.00
2	x2	>=			0.00
3	x3	>=			0.00
4	x4	>=			0.00

← Come back

Đơn hình Solve

Hình 5.5.1: Thông báo bài toán xoay vòng khi chọn thuật toán đơn hình để giải

Giá trị tối ưu: -1.0000

Nghiệm tối ưu:

Biến	Giá trị
x1	1.0000
x2	0.0000
x3	1.0000
x4	0.0000

Các bước thực thi: Từ vựng 1 (Khởi tạo) Từ vựng 2 Từ vựng 3 Từ vựng 4 Từ vựng 5 Từ vựng 6 Từ vựng 7 Từ vựng 8

Từ vựng 8:

→ Kết luận: Đã đạt từ vựng tối ưu. Dừng thuật toán.

* Giải thích cách đọc nghiệm và giá trị tối ưu: Để suy ra nghiệm tối ưu, ta cho tất cả các biến không cơ sở (các biến nằm ở vế phải) bằng 0, khi đó các biến cơ sở (vế trái) sẽ nhận giá trị bằng đúng hằng số tự do của phương trình tương ứng.

Đối với hàm mục tiêu, giá trị nhỏ nhất của Z chính là hằng số tự do trong phương trình Z hiện tại.

$$Z = -1.00 + 30.00 x_2 + 42.00 x_4 + 18.00 w_2 + 1.00 w_3$$

$$w_1 = 2.00 - 2.00 x_2 - 4.00 x_4 + 5.00 w_2 - 2.00 w_3$$

$$x_1 = 1.00 - 1.00 w_3$$

$$x_3 = 1.00 - 3.00 x_2 + 2.00 x_4 + 2.00 w_2 - 1.00 w_3$$

Biểu diễn hình học (Chỉ hỗ trợ cho trường hợp 2 biến số)

Hỏi Đáp OK

Hình 5.5.2: Bài toán xoay vòng được giải khi ta chọn thuật toán Bland để giải

Ta xét bài toán quy hoạch tuyến tính xoay vòng sau:

$$\begin{aligned} \min \quad & -10x_1 + 57x_2 + 9x_3 - 24x_4 - 100x_5 \\ & 0.5x_1 - 5.5x_2 - 2.5x_3 + 9x_4 + x_5 \leq 1 \\ & 0.5x_1 - 1.5x_2 - 0.5x_3 + x_4 + x_5 \leq 1 \\ & x_1 \leq 1 \\ & x_5 \leq 1 \\ & x_1, x_2, x_3, x_4, x_5 \geq 0 \end{aligned}$$

Số biến:

Số ràng buộc:

Nhập vào các hệ số của hàm mục tiêu:

	x0	x1	x2	x3	x4	x5
	0.00	-10.00	57.00	9.00	24.00	-100.00

Nhập vào các ràng buộc:

	x1	x2	x3	x4	x5	Sign	b. i
1	0.50	-5.50	-2.50	9.00	1.00	<=	1.00
2	0.50	-1.50	-0.50	1.00	1.00	<=	1.00
3	1.00	0.00	0.00	0.00	0.00	<=	1.00
4	0.00	0.00			1.00	<=	1.00

Cảnh báo!

Mẫu tương xoay vòng (Cycling)!
Thuật toán bị lặp vô hạn.

Nhập vào các ràng buộc cho biến:

	x1		
1	x1	>=	0.00
2	x2	>=	0.00
3	x3	>=	0.00
4	x4	>=	0.00
5	x5	>=	0.00

← Come back History Đơn hình Solve

Hình 5.5.3: Thông báo bài toán xoay vòng khi chọn thuật toán đơn hình để giải

Kiểm tra tính toán

Giá trị tối ưu:

-101.0000

Nghiệm tối ưu:

Biến	Giá trị
x1	1.0000
x2	0.0000
x3	1.0000
x4	0.0000
x5	1.0000

Các bước thực thi:

[w, x, z] HIỂN THỊ DẠNG TỰ VƯỢNG

HIỂN THỊ DẠNG BẢNG

Từ vựng 1 (Khởi tạo)

Từ vựng 2

Từ vựng 3

Từ vựng 4

Từ vựng 5

Từ vựng 6

Từ vựng 7

phương trình tương ứng:

Đối với hàm mục tiêu, giá trị nhỏ nhất của Mục Z chính là hằng số tự do trong phương trình Z hiện tại.

$$Z = -101.00 + 30.00 \ x2 + 42.00 \ x4 + 18.00 \ w2 + 1.00 \ w3 + 82.00 \ w4$$

$$x5 = 1.00 - 1.00 \ w4$$

$$x3 = 1.00 - 3.00 \ x2 + 2.00 \ x4 + 2.00 \ w2 - 1.00 \ w3 - 2.00 \ w4$$

$$v1 = 1.00 - 1.00 \ w3$$

Biểu diễn hình học (Chỉ hỗ trợ cho trường hợp 2 biến số)

Hỏi Đáp

OK

Hình 5.5.4: Bài toán xoay vòng được giải khi ta chọn thuật toán Bland để giải

6. Tổng kết và hướng phát triển

6.1 Kết quả đạt được

Sau quá trình nghiên cứu và xây dựng, nhóm đã phát triển thành công phần mềm hỗ trợ giải bài toán Quy hoạch tuyến tính với giao diện trực quan và thân thiện với người dùng. Phần mềm cho phép nhập bài toán dưới nhiều dạng khác nhau, tự động chuẩn hóa dữ liệu và áp dụng các thuật toán phù hợp để tìm nghiệm tối ưu.

Hệ thống đã cài đặt thành công phương pháp Đơn hình (Simplex Method), quy tắc Bland và thuật toán Hai pha, đồng thời hỗ trợ xử lý các trường hợp biến tự do và biến không dương. Bên cạnh việc xác định nghiệm tối ưu và giá trị tối ưu của hàm mục tiêu, phần mềm còn có khả năng hiển thị chi tiết từng bước thực hiện của thuật toán dưới dạng từ vựng, giúp người dùng dễ dàng theo dõi quá trình giải.

Ngoài ra, chương trình còn nhận diện và xử lý được nhiều trường hợp đặc biệt của bài toán quy hoạch tuyến tính như bài toán vô nghiệm, bài toán không giới nội, bài toán có vô số nghiệm tối ưu và hiện tượng xoay vòng. Đối với các bài toán có vô số nghiệm tối ưu, phần mềm có thể xác định thêm một nghiệm tối ưu khác để hỗ trợ người dùng nghiên cứu tập nghiệm tối ưu.

Trong quá trình kiểm thử, phần mềm đã được đối chiếu với nhiều bài toán giải bằng phương pháp thủ công. Kết quả thu được hoàn toàn phù hợp với các nghiệm và giá trị tối ưu theo lý thuyết, cho thấy tính chính xác và độ tin cậy của chương trình.

6.2 Hạn chế và hướng phát triển

Mặc dù đã đáp ứng được các yêu cầu cơ bản của đề tài, phần mềm vẫn còn một số hạn chế nhất định. Hiện tại chương trình chủ yếu tập trung vào các bài toán quy hoạch tuyến tính và chưa hỗ trợ các dạng bài toán tối ưu hóa nâng cao khác. Khả năng nhập dữ liệu vẫn chủ yếu thông qua giao diện chương trình nên chưa thực sự thuận tiện đối với các bài toán có kích thước lớn.

Trong tương lai, nhóm dự định tiếp tục phát triển phần mềm theo các hướng sau. Trước hết, hệ thống sẽ được bổ sung chức năng lưu và đọc dữ liệu từ các tệp Excel hoặc CSV nhằm giúp người dùng quản lý và trao đổi dữ liệu thuận tiện hơn. Bên cạnh đó, giao diện trực quan hóa sẽ được cải thiện để hỗ trợ tốt hơn việc quan sát quá trình giải và phân tích kết quả.

Ngoài các bài toán quy hoạch tuyến tính, nhóm cũng định hướng mở rộng phần mềm sang các bài toán tối ưu hóa khác, đặc biệt là Quy hoạch nguyên thông qua thuật toán Branch and Bound. Đây là hướng phát triển quan trọng nhằm nâng cao khả năng ứng dụng của phần mềm trong các bài toán thực tế có yêu cầu biến nguyên.

Với những kết quả đã đạt được, phần mềm có thể được sử dụng như một công cụ hỗ trợ học tập và nghiên cứu hiệu quả trong môn Quy hoạch tuyến tính, đồng thời tạo nền tảng cho việc phát triển các hệ thống tối ưu hóa phức tạp hơn trong tương lai.

7. Tài liệu tham khảo

- [1] Phan Quốc Khánh, Trần Huệ Nương, *Quy hoạch tuyến tính – Giáo trình hoàn chỉnh: Lý thuyết cơ bản, Phương pháp đơn hình, Bài toán mạng, Thuật toán điểm trong*, Nhà xuất bản Giáo dục.
- [2] Tài liệu bài giảng môn *Quy hoạch tuyến tính* của giảng viên phụ trách học phần.
- [3] Các bài tập, ví dụ và tài liệu tham khảo được cung cấp trong quá trình học môn Quy hoạch tuyến tính.

8. Hướng dẫn tải và cài đặt phần mềm

8.1 Giới thiệu chung

Phần mềm Giải Quy Hoạch Tuyến Tính (PhanMemQHTT) được phát triển bởi nhóm sinh viên Trường Đại học Khoa học Tự nhiên – ĐHQG-HCM (HCMUS). Phần mềm hỗ trợ giải các bài toán quy hoạch tuyến tính một cách trực quan và dễ sử dụng.

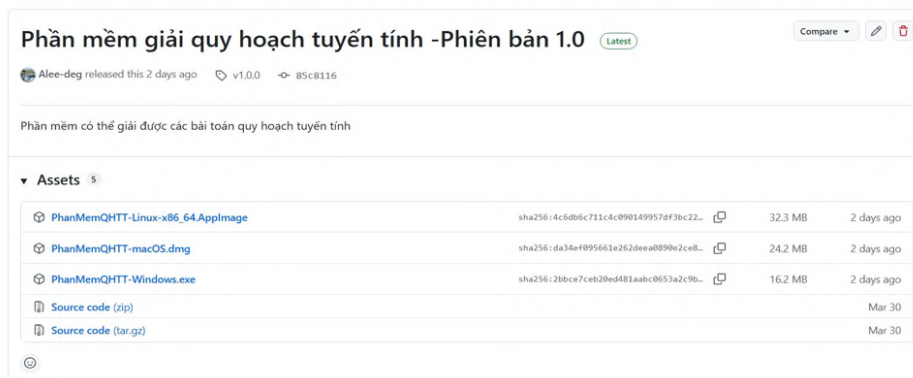
Tài liệu này hướng dẫn cách tải và cài phần mềm trên ba hệ điều hành: Windows, macOS và Linux.

8.2 Quy trình cài đặt phần mềm

8.2.1 Tải phần mềm từ GitHub

Truy cập trang GitHub: https://github.com/Alee-deg/SolveLinearProgramming/releases/tag/Alee_LH. Chọn phiên bản mới nhất (có nhãn “Latest”). Trong phần Assets, tải file tương ứng với hệ điều hành của bạn:

- **Windows:** PhanMemQHTT-Windows.exe
- **macOS:** PhanMemQHTT-macOS.dmg
- **Linux:** PhanMemQHTT-Linux-amd64.deb (khuyến nghị) hoặc PhanMemQHTT-Linux-x86_64.AppImage (bản portable)



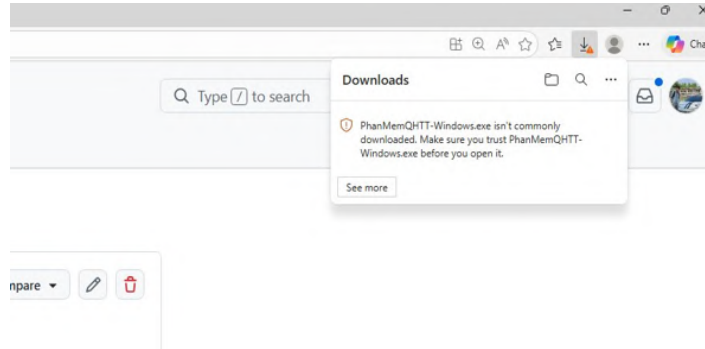
Hình 8.2.1: Trang GitHub Releases — chọn file phù hợp với hệ điều hành

8.2.2 Đối với hệ điều hành Windows

Bước 1: Xử lý cảnh báo trình duyệt Edge

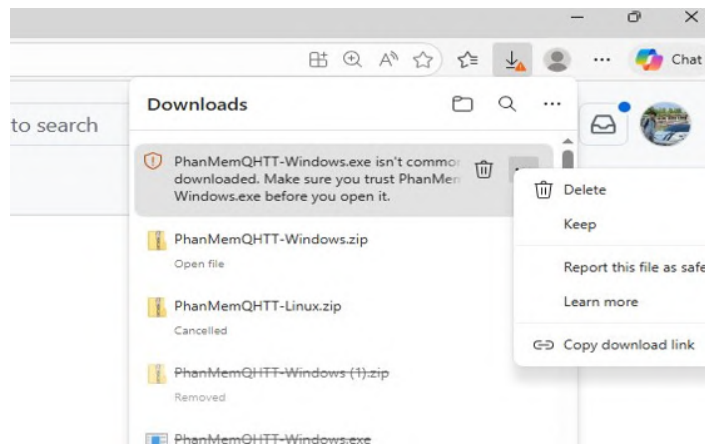
Vì phần mềm chưa được ký số (code signing), trình duyệt Microsoft Edge sẽ hiện cảnh báo khi tải. Đây là hiện tượng bình thường với phần mềm mới phát hành.

Bước 1.1: Nhấn dấu ba chấm nằm ở góc trên bên phải trên thanh Downloads
Sau khi nhấn tải, thanh Downloads sẽ hiện cảnh báo "PhanMemQHTT-Windows.exe isn't commonly downloaded". Nhấn vào dấu ba chấm nằm ở góc trên bên phải để xem thêm tùy chọn.



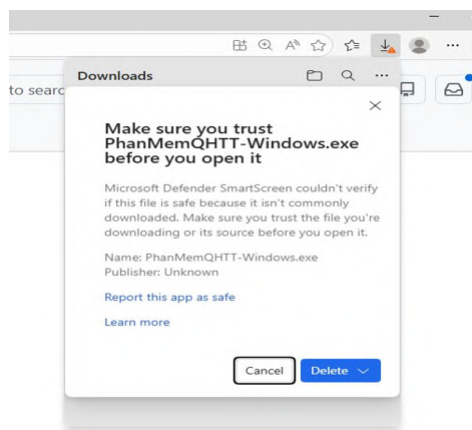
Hình 8.2.2: Cảnh báo xuất hiện ở thanh Downloads

Bước 1.2: Chọn "Keep" từ menu
Trong menu xuất hiện, chọn "Keep" để tiếp tục giữ file tải về.

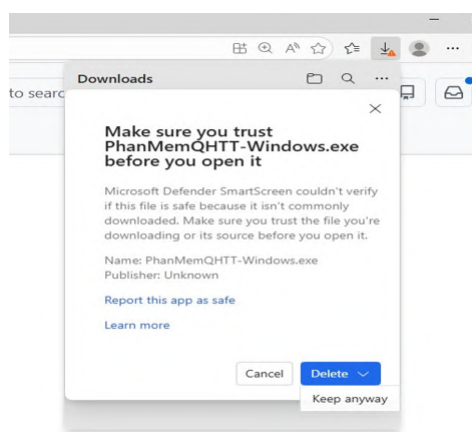


Hình 8.2.3: Chọn "Keep" để tiếp tục

Bước 1.3: Xác nhận "Keep anyway"
Trình duyệt hiện hộp thoại xác nhận của Microsoft Defender SmartScreen. Nhấn vào mũi tên bên cạnh nút "Delete" rồi chọn "Keep anyway".



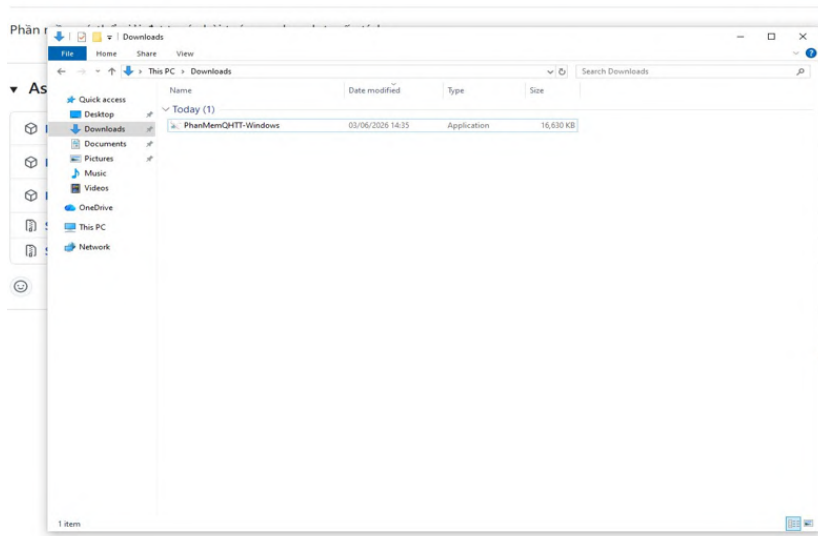
Hình 8.2.4: Hộp thoại xác nhận SmartScreen



Hình 8.2.5: Chọn "Keep anyway" để lưu file

Bước 2: **Mở và chạy phần mềm**

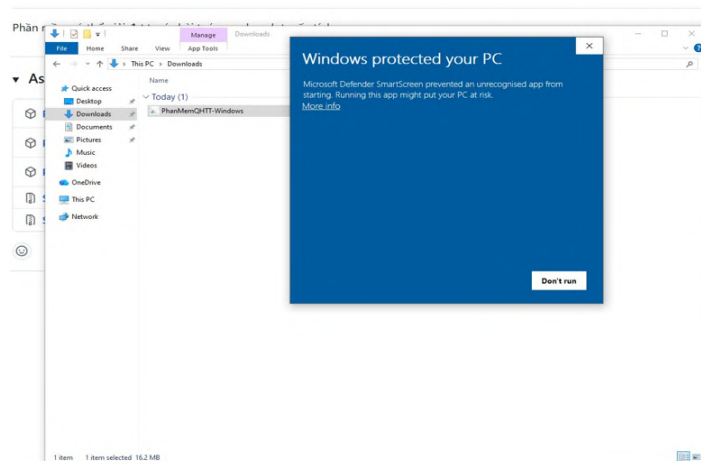
Mở thư mục Downloads, tìm file PhanMemQHTT-Windows.exe và nhấn đúp chuột để chạy.



Hình 8.2.6: File PhanMemQHTT-Windows.exe trong thư mục Downloads

Bước 2.1: Xử lý cảnh báo Windows SmartScreen

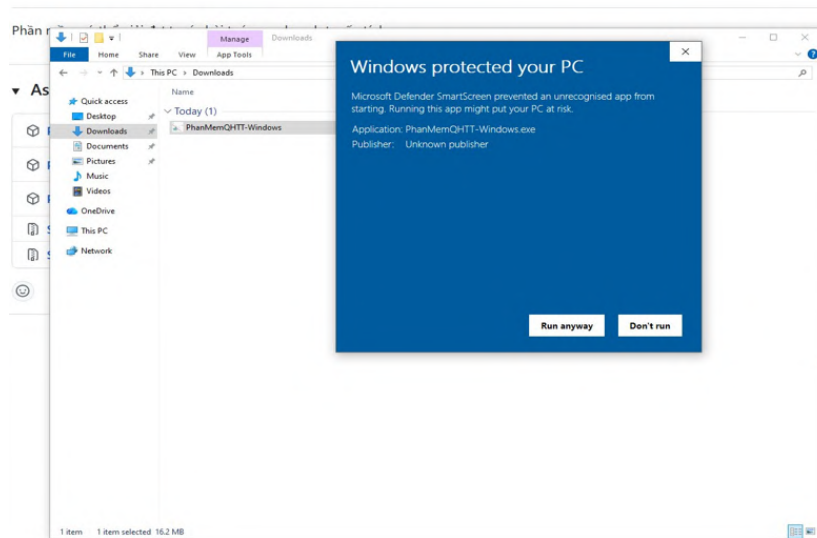
Màn hình "Windows protected your PC" sẽ xuất hiện. Nhấn "More info" để hiện thêm tùy chọn.



Hình 8.2.7: Màn hình SmartScreen — nhấn More info

Bước 2.2: Nhấn "Run anyway"

Sau khi nhấn "More info", thông tin ứng dụng hiện ra. Nhấn "Run anyway" để khởi chạy phần mềm.



Hình 8.2.8: Nhấn "Run anyway" để chạy phần mềm

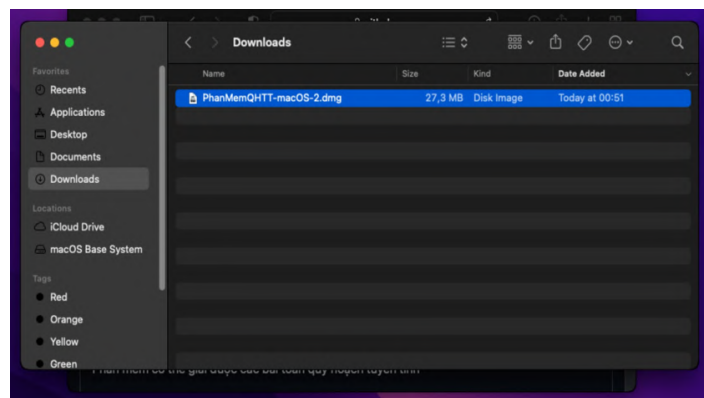
Lưu ý: Chỉ thực hiện bước này khi tải từ trang GitHub chính thức của dự án.

Từ lần chạy thứ hai trở đi, Windows sẽ không hiện cảnh báo này nữa.

8.2.3 Đối với hệ điều hành MacOS

Bước 1: Mở file DMG

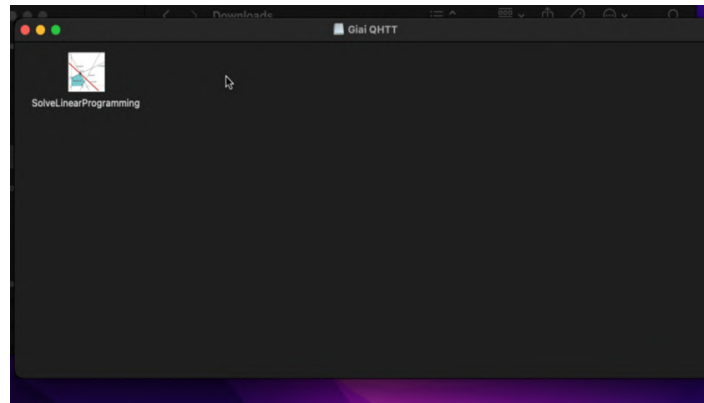
Sau khi tải xong, mở thư mục Downloads. File PhanMemQHTT-macOS.dmg sẽ xuất hiện nhấn đúp để mở file DMG.



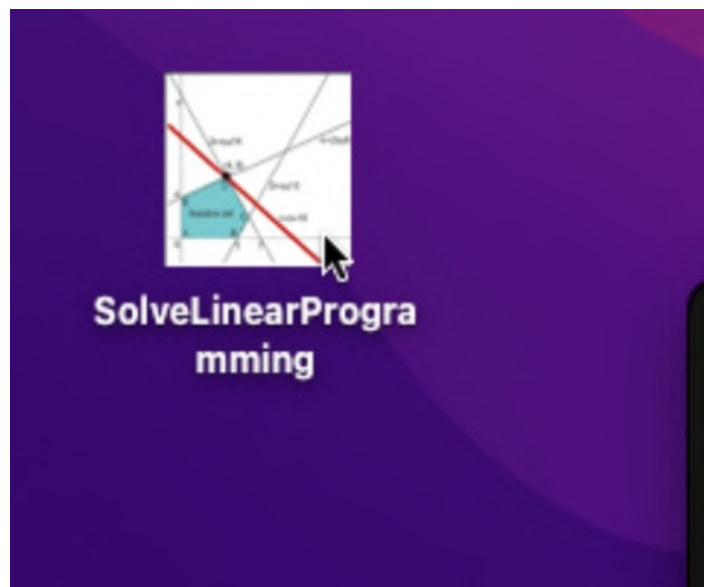
Hình 8.2.9: File PhanMemQHTT-macOS.dmg trong thư mục Downloads

Bước 2: Copy ứng dụng vào Desktop

Sau khi mở DMG, một cửa sổ "Giai QHTT" sẽ xuất hiện chứa file SolveLinearProgramming. Kéo thả ứng dụng này ra Desktop hoặc thư mục Applications.



Hình 8.2.10: Cửa sổ DMG chứa ứng dụng SolveLinearProgramming



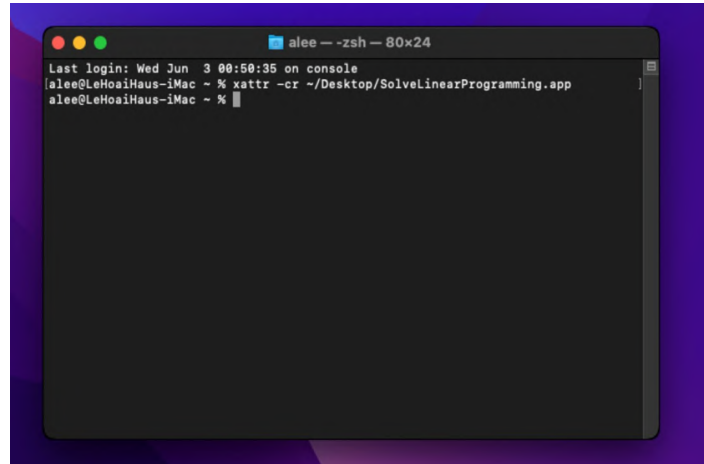
Hình 8.2.11: Icon ứng dụng SolveLinearProgramming trên Desktop

Bước 3: Gỡ cảnh báo Gatekeeper bằng Terminal

macOS sẽ chặn ứng dụng chưa được ký số. Để mở được ứng dụng, cần chạy lệnh sau trong Terminal:

```
xattr -cr ~/Desktop/SolveLinearProgramming.app
```

Mở Terminal (trong Finder > Applications > Utilities > Terminal hoặc tìm trong Spotlight), dán lệnh trên và nhấn Enter.



Hình 8.2.12: Chạy lệnh xattr -cr trong Terminal để gỡ cảnh báo Gatekeeper

Bước 4: Khởi chạy ứng dụng

Sau khi chạy lệnh xattr thành công, nhấn đúp vào icon SolveLinearProgramming trên Desktop. Phần mềm sẽ mở ra với giao diện chính.



Hình 8.2.13: Giao diện chính của phần mềm trên macOS

- Nhấn "Nhấn vào đây để bắt đầu" để bắt đầu sử dụng phần mềm.
- Từ lần chạy thứ hai trở đi, chỉ cần click vào icon của ứng dụng không cần lặp lại các bước trên.

8.2.4 Đối với hệ điều hành Linux

Cách 1: Cài đặt bằng file .deb (khuyến nghị)

Trong phần Assets của GitHub Releases, tải file PhanMemQHTT-Linux-amd64.deb. Đây là cách cài đặt khuyến nghị cho Ubuntu, Debian và Linux Mint vì sau khi cài, phần mềm sẽ xuất hiện trong menu ứng dụng Linux và có icon như một ứng dụng thông thường.

Bước 1.1: Mở Terminal tại thư mục chứa file .deb

Sau khi tải file về, mở thư mục Downloads, nhấn chuột phải vào vùng trống và chọn Open in Terminal. Nếu file nằm trong thư mục khác, hãy mở Terminal tại đúng thư mục đang chứa file .deb.

Bước 1.2 Chạy lệnh cài đặt

Nhập lệnh sau rồi nhấn Enter. Hệ thống có thể yêu cầu nhập mật khẩu máy tính để xác nhận cài đặt:

```
sudo apt install ./PhanMemQHTT-Linux-amd64.deb
```

Khi quá trình cài đặt hoàn tất, nếu Terminal hiển thị dòng “Setting up phanmemqhtt (1.0.0) ...” nghĩa là phần mềm đã được cài thành công.

Lưu ý: Nếu xuất hiện cảnh báo “Download is performed unsandboxed as root ... Permission denied”, đây chỉ là cảnh báo quyền đọc file trong thư mục Downloads, không phải lỗi cài đặt nếu phần mềm vẫn được “Setting up” thành công.

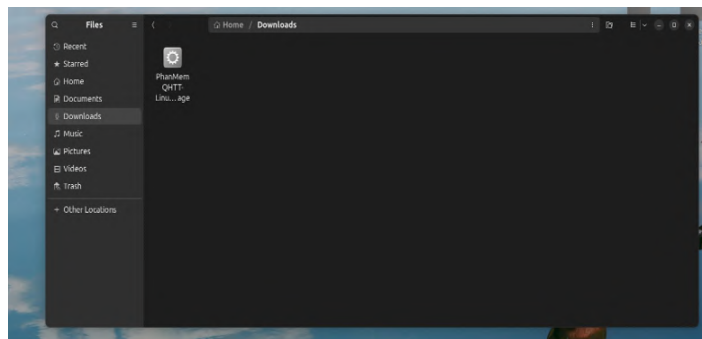
Bước 1.3 Mở phần mềm sau khi cài đặt Sau khi cài đặt, nhấn phím Super/Windows trên bàn phím rồi tìm “Phần mềm Giải Quy Hoạch Tuyến Tính”. Người dùng cũng có thể mở phần mềm bằng Terminal với lệnh:

```
phanmemqhtt
```

- * Cách cài bằng file .deb giúp phần mềm có icon trong menu ứng dụng và thuận tiện hơn so với chạy trực tiếp file AppImage.
- * Nếu cần gỡ cài đặt phần mềm, có thể dùng lệnh: `sudo apt remove phanmemqhtt`

Cách 2: Chạy bản AppImage portable (tùy chọn)

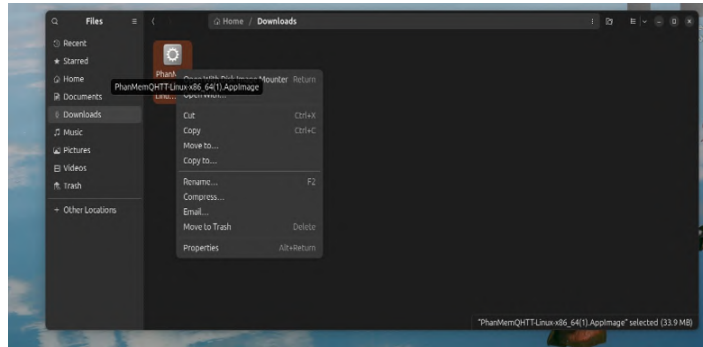
Nếu bạn không muốn cài đặt hoặc đang dùng bản Linux không thuộc nhóm Ubuntu/Debian/Linux Mint, có thể dùng bản portable PhanMemQHTT-Linux-x86-64.AppImage. Sau khi tải xong, mở thư mục Downloads trong trình quản lý file để cấp quyền thực thi và chạy trực tiếp.



Hình 8.2.14: File AppImage trong thư mục Downloads trên Linux

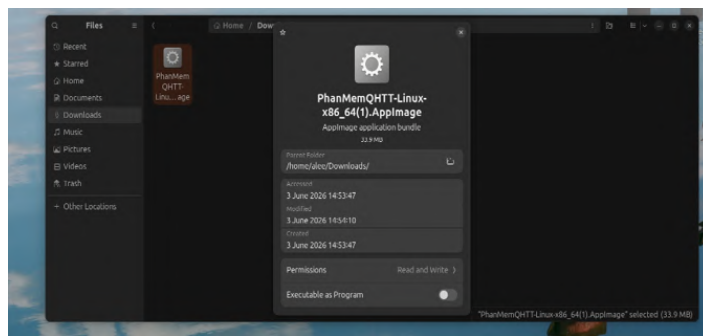
Bước 2.1: Mở Properties để cấp quyền thực thi

Trên Linux, file AppImage cần được cấp quyền thực thi (executable) trước khi chạy. Nhấn chuột phải vào file và chọn "Properties".

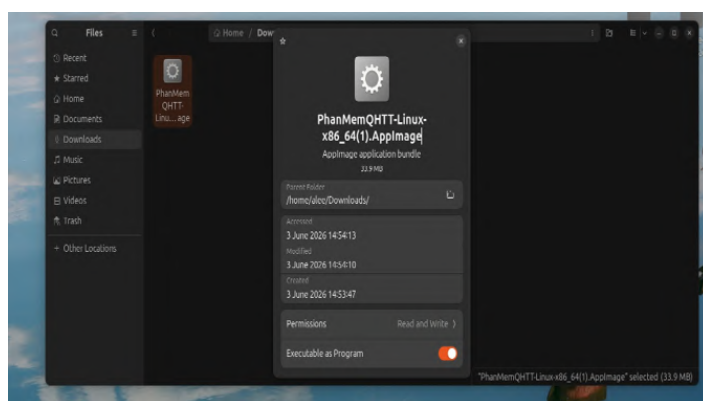


Hình 8.2.15: Nhấn chuột phải chọn Properties

Bước 2.2: **Bật "Executable as Program"** Trong cửa sổ Properties, tìm mục "Executable as Program" và bật toggle sang trạng thái ON (màu cam). Sau đó đóng cửa sổ Properties.



Hình 8.2.16: Toggle "Executable as Program" đang TẮT (trước khi bật)

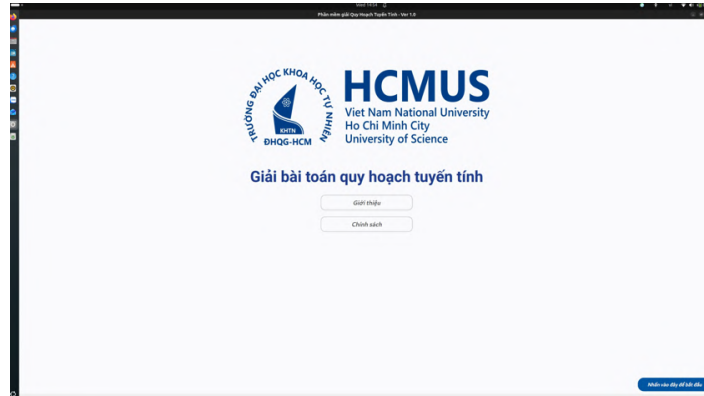


Hình 8.2.17: Toggle "Executable as Program" đã BẬT (màu cam)

Lưu ý: Nếu trình quản lý file không có tùy chọn này, bạn có thể cấp quyền bằng Terminal với lệnh: `chmod +x /Downloads/PhanMemQHTT-Linux-x86_64.AppImage`

Bước 2.3 Khởi chạy ứng dụng

Sau khi cấp quyền thực thi, nhấn đúp chuột vào file AppImage để khởi động. Phần mềm sẽ chạy trực tiếp, không cần cài đặt thêm. Nếu muốn ứng dụng xuất hiện trong menu Linux và có icon như ứng dụng thông thường, nên dùng file đuôi deb ở Bước 1 của hệ điều hành Linux.



Hình 8.2.18: Giao diện chính của phần mềm trên Linux

- * Nhấn "Nhấn vào đây để bắt đầu" để bắt đầu sử dụng phần mềm.
- * Nếu cài bằng file .deb, từ lần sau chỉ cần mở phần mềm từ menu ứng dụng Linux. Nếu dùng AppImage, hãy mở lại file AppImage đã được cấp quyền thực thi.

8.3 Hướng dẫn cài đặt API Key cho chức năng Chatbot

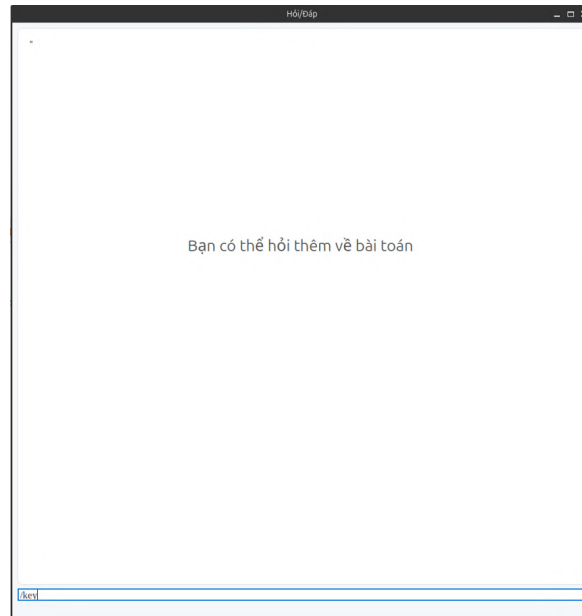
Chức năng Hỏi/Đáp Chatbot cho phép người dùng đặt câu hỏi liên quan đến bài toán quy hoạch tuyến tính đang giải. Để sử dụng chức năng này, người dùng cần tạo API Key trên Groq Console và dán key vào phần mềm.

8.3.1 Mở cửa sổ Hỏi/Đáp trong phần mềm

Sau khi giải bài toán và chuyển sang cửa sổ kết quả, nhấn nút Hỏi/Đáp để mở cửa sổ chatbot. Trong ô nhập ở phía dưới, gõ lệnh sau rồi nhấn Enter:

`/key`

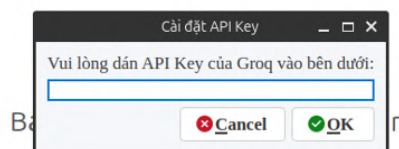
Lệnh này dùng để mở hộp thoại cài đặt API Key cho chatbot.



Hình 8.3.1: Nhập lệnh /key trong cửa sổ Hỏi Đáp

8.3.2 Dán API Key vào hộp thoại cài đặt

Sau khi nhập lệnh /key, phần mềm sẽ hiển thị hộp thoại “Cài đặt API Key”. Người dùng sẽ dán API Key của Groq vào ô trống này sau khi tạo key ở các bước tiếp theo.

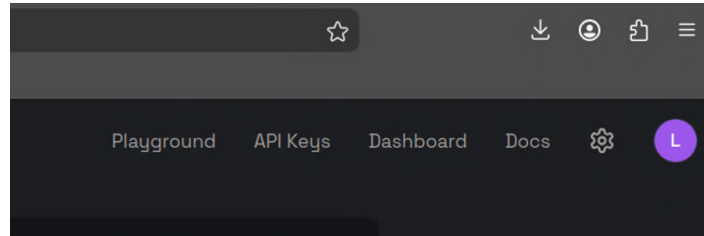


Hình 8.3.2: Hộp thoại cài đặt API Key của Groq trong phần mềm

8.3.3 Truy cập Groq Console

Mở trình duyệt và truy cập địa chỉ: <https://console.groq.com/> Đăng nhập hoặc tạo tài khoản Groq nếu chưa có. Sau khi đăng nhập thành công, giao diện Groq

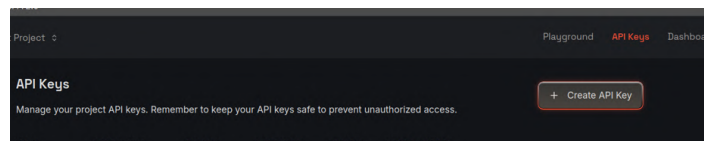
Console sẽ hiển thị các mục như Playground, API Keys, Dashboard và Docs.



Hình 8.3.3: Giao diện Groq Console sau khi đăng nhập

8.3.4 Vào mục API Keys

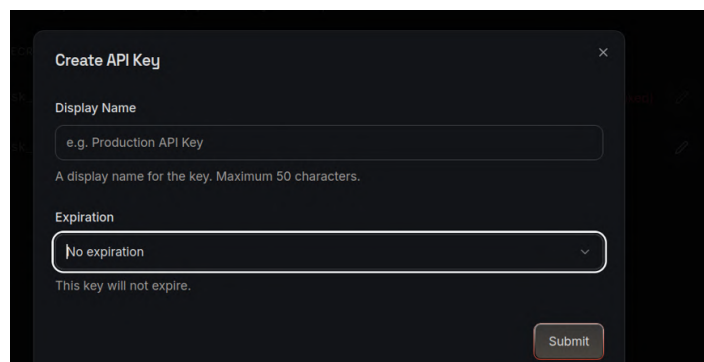
Trên thanh điều hướng của Groq Console, chọn mục API Keys. Đây là nơi dùng để quản lý và tạo khóa API cho tài khoản Groq. Tại màn hình API Keys, nhấn nút Create API Key để tạo khóa mới.



Hình 8.3.4: Chọn API Keys và nhấn Create API Key

8.3.5 Tạo API Key mới

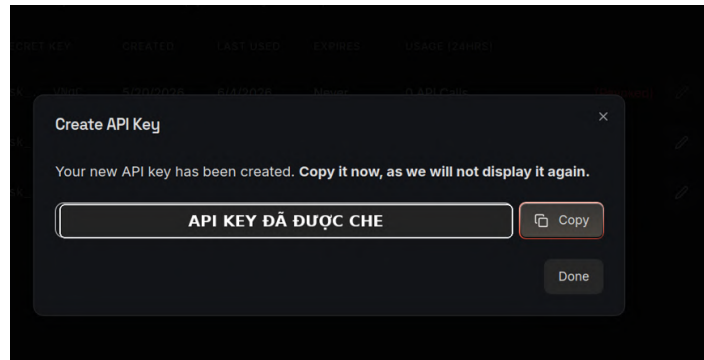
Trong hộp thoại Create API Key, nhập tên hiển thị cho key ở ô Display Name. Nên đặt tên dễ nhớ, ví dụ: **PhanMemQHTT-Chatbot**. Ở mục Expiration, có thể chọn No expiration nếu muốn key không hết hạn, hoặc chọn thời hạn phù hợp nếu muốn kiểm soát bảo mật tốt hơn. Sau đó nhấn Submit để tạo key.



Hình 8.3.5: Nhập Display Name, chọn Expiration và nhấn Submit

8.3.6 Sao chép API Key

Sau khi tạo thành công, Groq sẽ hiển thị API Key mới. Người dùng cần nhấn Copy và lưu lại ngay, vì key chỉ được hiển thị một lần.



Hình 8.3.6: Sao chép API Key mới tạo. Trong tài liệu này, key đã được che để đảm bảo bảo mật

8.3.7 Dán API Key vào phần mềm

Quay lại hộp thoại “Cài đặt API Key” trong phần mềm, dán API Key vừa sao chép vào ô nhập liệu, sau đó nhấn OK. Sau khi lưu key thành công, phần mềm có thể sử dụng API này để gửi câu hỏi tới Groq và nhận phản hồi từ chatbot.

8.3.8 Kiểm tra chức năng Chatbot

Sau khi cài đặt API Key, nhập một câu hỏi liên quan đến bài toán đang giải trong cửa sổ Hỏi/Đáp. Ví dụ:

- Vì sao bài toán này có nghiệm tối ưu?
- Giải thích cách đọc nghiệm tối ưu trong bảng đơn hình.
- Vì sao biến x_1 được chọn làm biến vào?
- Vì sao bài toán bị vô nghiệm hoặc không giới nội?
- Giải thích dạng từ vựng hiện tại của bài toán.

Chatbot được thiết kế để hỗ trợ giải thích các nội dung liên quan đến quy hoạch tuyến tính, phương pháp đơn hình, nghiệm tối ưu và các bước biến đổi trong quá trình giải.

8.4 Một số lỗi thường gặp khi cài API Key

8.4.1 Chatbot không phản hồi

Nguyên nhân có thể là máy tính chưa có Internet, API Key nhập sai, API Key đã bị thu hồi, hoặc tài khoản Groq chưa đủ điều kiện sử dụng API. Cách xử lý: kiểm

tra lại kết nối mạng, tạo lại API Key mới, dán lại key vào phần mềm và khởi động lại cửa sổ Hỏi/Đáp.

8.4.2 Nhập /key nhưng không hiện hộp thoại

Người dùng cần kiểm tra đã nhập đúng lệnh /key ở ô nhập phía dưới cửa sổ Hỏi/Đáp hay chưa. Sau khi nhập xong cần nhấn Enter.

8.4.3 API Key bị lộ hoặc không còn sử dụng được

Nếu API Key bị lộ hoặc không còn sử dụng được, hãy truy cập Groq Console, vào mục API Keys, thu hồi key cũ và tạo key mới. Sau đó quay lại phần mềm, nhập lại lệnh /key và dán key mới vào.

8.5 Hoàn tất cài đặt Chatbot

Sau khi API Key được lưu thành công, chức năng Hỏi/Đáp Chatbot đã sẵn sàng sử dụng. Người dùng có thể đặt câu hỏi để được hỗ trợ giải thích bài toán, cách đọc bảng kết quả, dạng từ vựng, nghiệm tối ưu và các trường hợp đặc biệt trong quy hoạch tuyến tính.

Nếu gặp bất kỳ vấn đề nào trong quá trình cài đặt, vui lòng liên hệ nhóm phát triển qua trang GitHub của dự án hoặc gửi báo cáo lỗi (issue) trực tiếp trên repository.

9. Hướng dẫn sử dụng

9.1 Hướng dẫn thao tác từng bước

9.1.1 Bước 1: Bắt đầu sử dụng

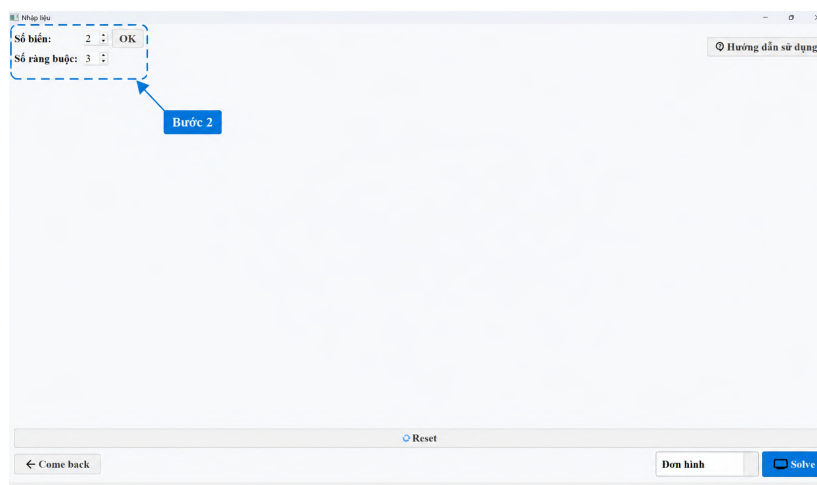
Nhấn vào nút để bắt đầu truy cập vào giao diện chính của phần mềm.



Hình 9.1.1: Màn hình bắt đầu

9.1.2 Bước 2: Khởi tạo kích thước bài toán

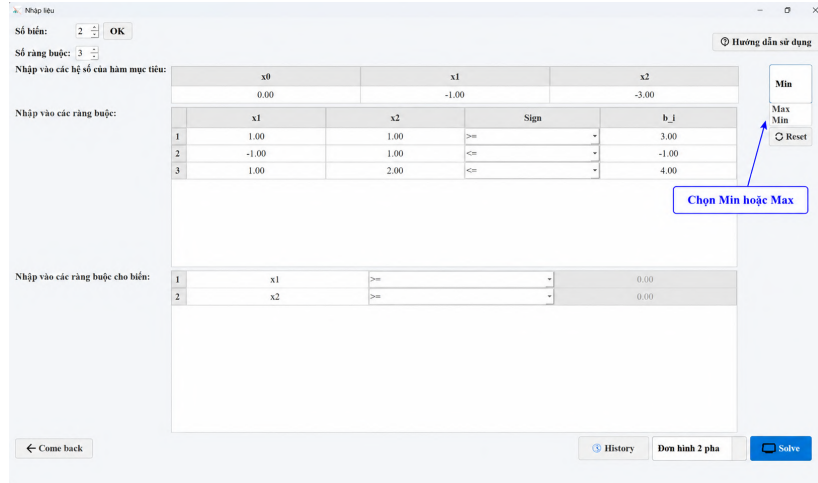
Nhập số lượng biến và số lượng ràng buộc của bài toán, sau khi điền xong nhấn OK.



Hình 9.1.2: Khởi tạo kích thước

9.1.3 Bước 3: Nhập dữ liệu bài toán

Tiến hành nhập các hệ số của hàm mục tiêu và các hệ số tương ứng của các ràng buộc.



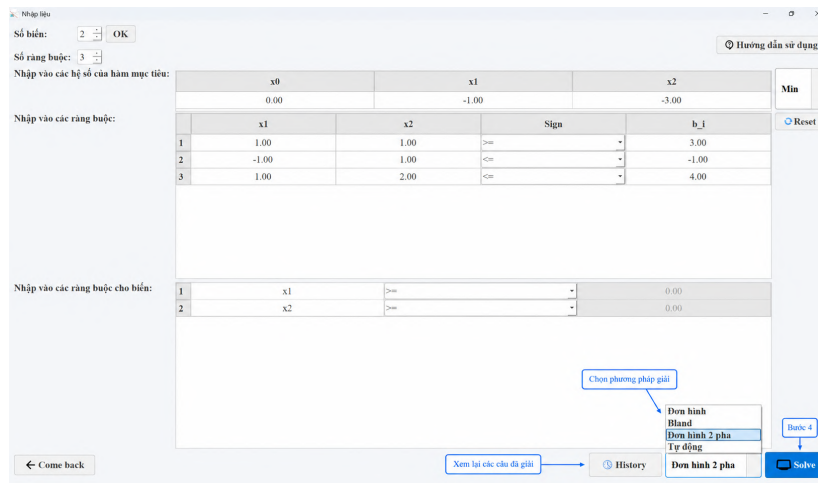
Hình 9.1.3: Giao diện nhập liệu

9.1.4 Bước 4: Chọn phương pháp và giải bài toán

Lựa chọn mô hình giải phù hợp từ menu:

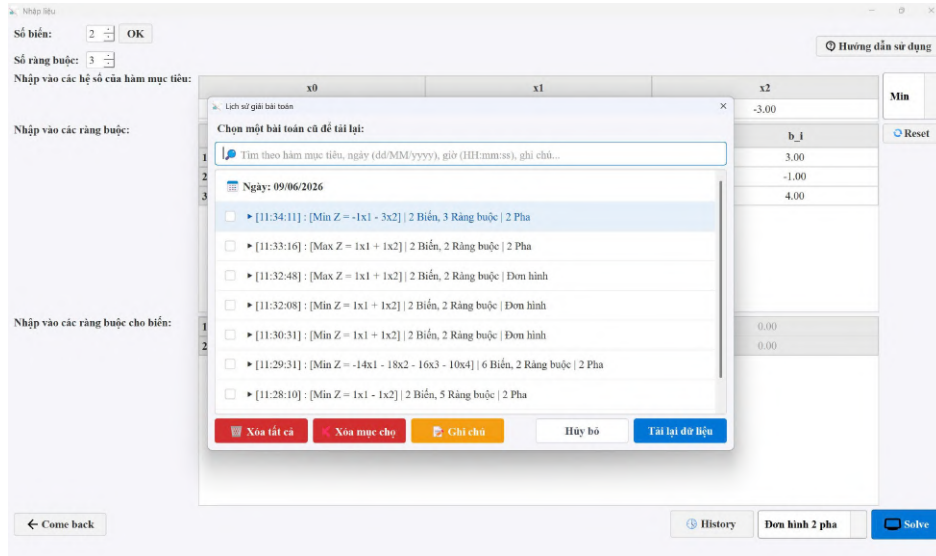
- **Lưu ý:** Nếu nhập bài toán chỉ có thể giải bằng phương pháp 2 pha, khi bạn chọn mô hình “Đơn hình” hoặc “Bland” thì phần mềm sẽ thông báo lỗi, và ngược lại.
- “Tự động” là chế độ mà phần mềm cho là hợp lý và tối ưu nhất cho bài toán hiện tại.

Sau khi chọn xong, nhấn nút **Solve** để tiến hành giải.



Hình 9.1.4: Chọn mô hình giải

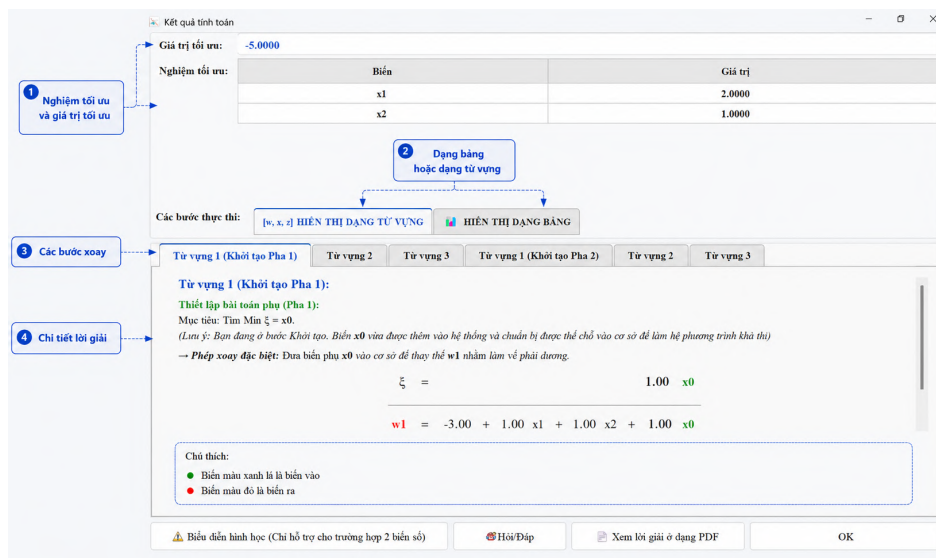
Bạn có thể nhấn vào **History** để xem lại các câu đã giải.



Hình 9.1.5: Xem lại các câu đã giải trước đây

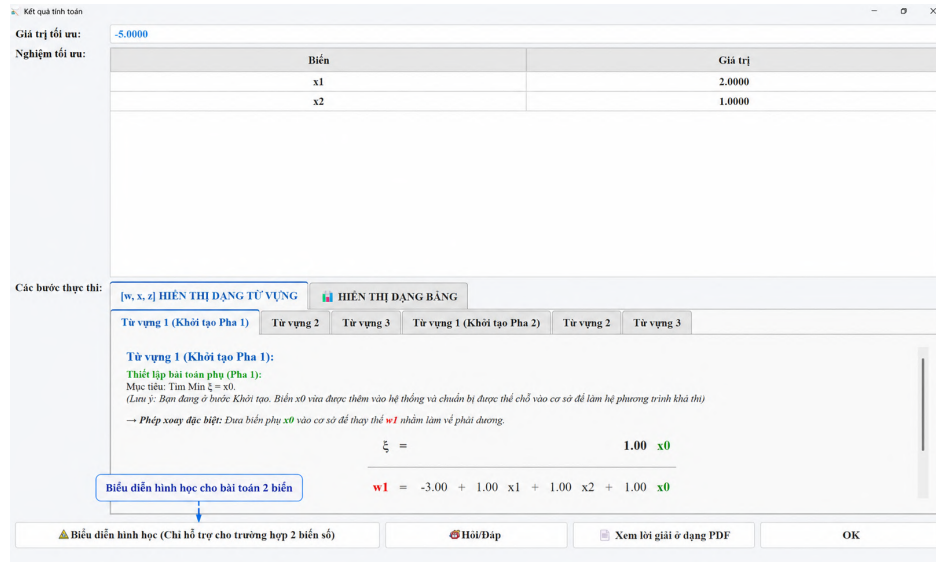
9.1.5 Bước 5: Hiện thị các bước giải và kết quả

Phần mềm sẽ tiến hành tính toán và xuất ra bảng lời giải chi tiết qua từng vòng lặp cho bài toán quy hoạch tuyến tính của bạn.

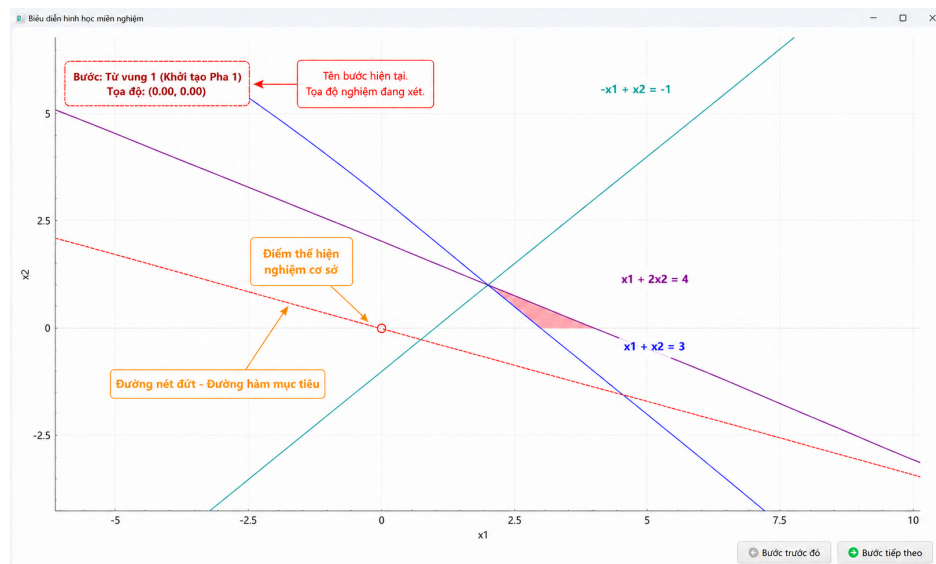


Hình 9.1.6: Kết quả tính toán chi tiết

9.1.6 Bước 6: Biểu diễn hình học cho bài toán 2 biến

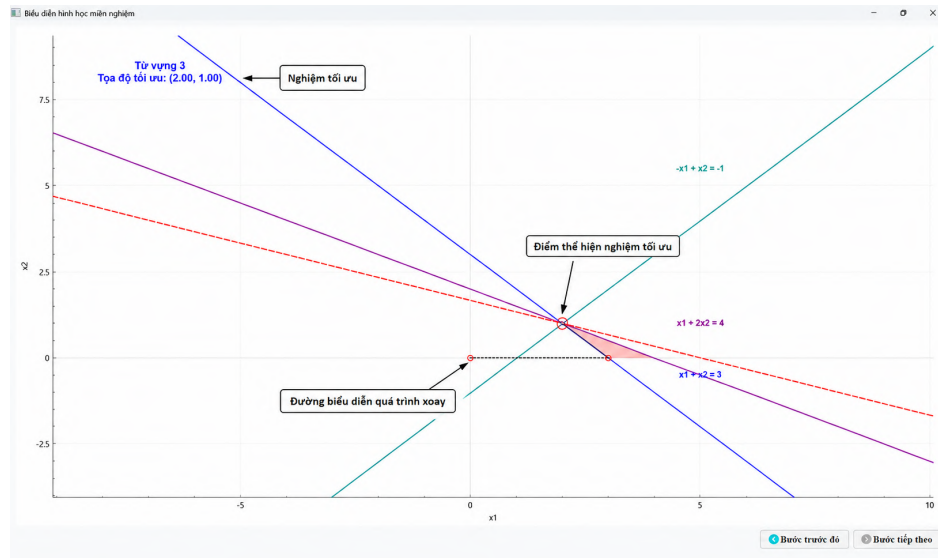


Hình 9.1.7: Biểu diễn hình học (1)



Hình 9.1.8: Biểu diễn hình học (2)

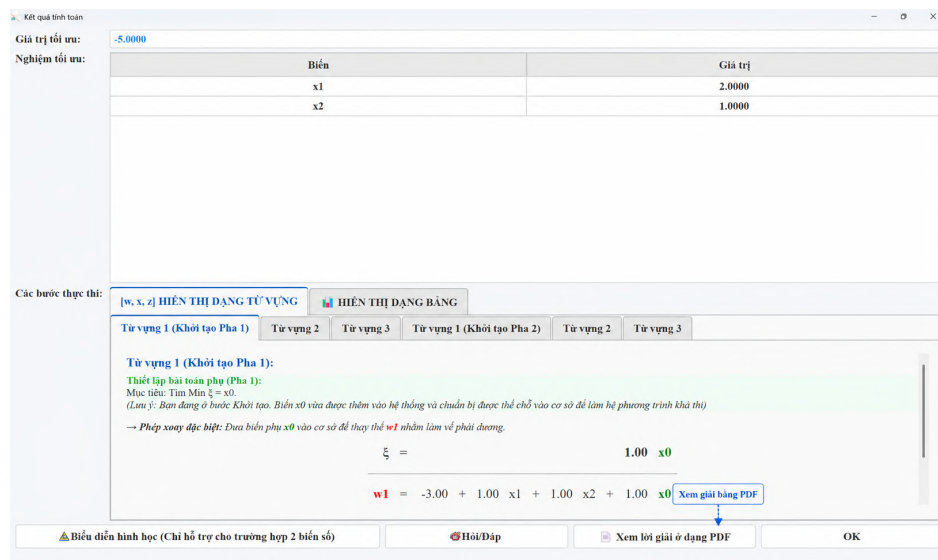
- Các đường thẳng biểu diễn các ràng buộc của bài toán.
- Vùng tô màu thể hiện miền nghiệm chấp nhận được.



Hình 9.1.9: Hiển thị kết quả tối ưu

9.1.7 Bước 7: Xem lời giải bằng file PDF

Phần mềm hỗ trợ người dùng xem lời giải ở dạng PDF và TEX.



Kết quả tính toán

Giá trị tối ưu: -5.0000

Nghiệm tối ưu:

Biến	Giá trị
x1	2.0000
x2	1.0000

Các bước thực thi:

[w, x, z] HIỂN THỊ DẠNG TỪ VỰNG HIỂN THỊ DẠNG BẢNG

Từ vùng 1 (Khởi tạo Pha 1) Từ vùng 2 Từ vùng 3 Từ vùng 1 (Khởi tạo Pha 2) Từ vùng 2 Từ vùng 3

Từ vùng 1 (Khởi tạo Pha 1):

Thiết lập bài toán phụ (Pha 1):

Mục tiêu: Tìm Min $\xi = x_0$.

(Lưu ý: Biến đang ở trước Khởi tạo. Biến x_0 vừa được thêm vào hệ thống và chuẩn bị được thể chế vào cơ sở để làm hệ phương trình khả thi)

→ **Phương pháp đặc biệt:** Đưa biến phụ x_0 vào cơ sở để thay thế w_1 nhằm làm vế phải dương.

$\xi = 1.00 \cdot x_0$

$w_1 = -3.00 + 1.00 \cdot x_1 + 1.00 \cdot x_2 + 1.00 \cdot x_0$

Xem giải bằng PDF

Biểu diễn hình học (Chỉ hỗ trợ cho trường hợp 2 biến số) Hỏi/Đáp Xem lời giải ở dạng PDF OK

Hình 9.1.10: Xem lời giải ở dạng PDF

Xem lời giải với PDF

GIẢI BÀI TOÁN QUY HOẠCH TUYẾN TÍNH

1. BÀI TOÁN GỐC

Hàm mục tiêu:

$$\text{Min } Z = -x_1 - 3.00x_2$$

Hệ ràng buộc:

$$\begin{aligned} x_1 + x_2 &\geq 3.00 \\ x_1 + x_2 &\leq -1.00 \\ x_1 + 2.00x_2 &\leq 4.00 \\ x_1 &\geq 0.00 \\ x_2 &\geq 0.00 \end{aligned}$$

2. CÁC BƯỚC GIẢI (DẠNG TỬ VỰNG)

Từ vựng 1 (Khởi tạo Pha 1)

→ Phép xoay đặc biệt: Đưa biến phụ x_0 vào cơ sở để thay thế w_1 nhằm làm về phải dương.

$$\xi =$$

	x_1	x_2	x_0				
$w_1 =$	-3.00	+	x_1	+	x_2	+	x_0
$w_2 =$	-1.00	+	x_1	-	x_2	+	x_0
$w_3 =$	4.00	-	x_1	-	2.00 x_2	+	x_0

Từ vựng 2

→ Chọn x_1 làm biến vào, đẩy x_0 ra khỏi cơ sở.

Tải xuống lời giải

Tải xuống PDF

Tải xuống .tex

Hình 9.1.11: Lời giải và tải về PDF hoặc TEX

9.1.8 Bước 8: Chatbot hỗ trợ giải đáp

Phần mềm có tích hợp Trợ lý ảo (Chatbot) để hỗ trợ người dùng khi gặp thắc mắc.

Kết quả tính toán

Giá trị tối ưu: -5.0000

Nghiệm tối ưu:

Biến	Giá trị
x_1	2.0000
x_2	1.0000

Các bước thực thi:

[w, x, z] HIỂN THỊ DẠNG TỬ VỰNG

HIỂN THỊ DẠNG BẢNG

Từ vựng 1 (Khởi tạo Pha 1)

Từ vựng 2

Từ vựng 3

Từ vựng 1 (Khởi tạo Pha 2)

Từ vựng 2

Từ vựng 3

Từ vựng 1 (Khởi tạo Pha 1):

Thiết lập bài toán phụ (Pha 1):

Mục tiêu: Tìm Min $\xi = x_0$.

(Lưu ý: Bạn đang ở bước Khởi tạo. Biến x_0 vừa được thêm vào hệ thống và chuẩn bị được thế chỗ vào cơ sở để làm hệ phương trình khả thi)

→ Phép xoay đặc biệt: Đưa biến phụ x_0 vào cơ sở để thay thế w_1 nhằm làm về phải dương.

$$\xi =$$

	x_1	x_2	x_0				
$w_1 =$	-3.00	+	1.00 x_1	+	1.00 x_2	+	1.00 x_0

Chatbot

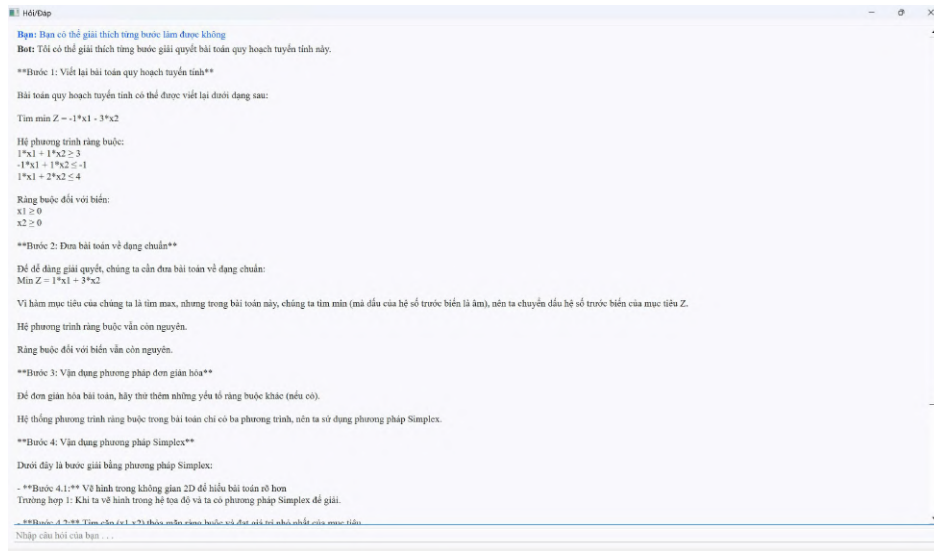
Biểu diễn hình học (Chỉ hỗ trợ cho trường hợp 2 biến số)

Hỏi/Đáp

Xem lời giải ở dạng PDF

OK

Hình 9.1.12: Chatbot tích hợp trong phần mềm



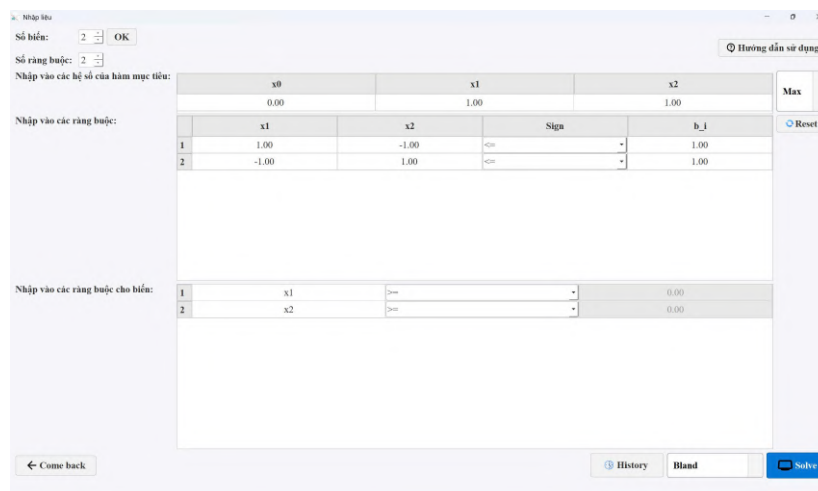
Hình 9.1.13: Thảo luận chi tiết với Chatbot

9.2 Xử lý các thông báo ngoại lệ

Trong quá trình giải, phần mềm có thể trả về một số thông báo thuật toán đặc biệt. Dưới đây là các trường hợp có thể xảy ra:

9.2.1 Trường hợp “Bài toán không giới nội”

Thuật toán phát hiện tập phương án không bị chặn, dẫn đến giá trị hàm mục tiêu có thể tiến tới vô cực.



Hình 9.2.1: Bài toán không giới nội

Kết quả tính toán

Giá trị tối ưu: $+\infty$ (Không giới nội) Bài toán không giới nội nên Max = $+\infty$

Nghiệm tối ưu:

Biến	Phương trình (Tập nghiệm vô cực)
x1	$1.00 + 1.00 x2 - 1.00 w1 \geq 0$
x2	Biến tham số tùy ý ≥ 0
w1	Biến tham số tùy ý ≥ 0
w2	$2.00 - 1.00 w1 \geq 0$

Tập nghiệm tổng quát của bài toán

Các bước thực thi:

[w, x, z] HIỂN THỊ DẠNG TỬ VỆNG HIỂN THỊ DẠNG BẢNG

Từ vựng 1 (Khởi tạo Pha 1) Từ vựng 1 (Khởi tạo Pha 2) Từ vựng 2

Từ vựng 2:

→ **Kết luận:** Bài toán không giới nội. Dừng thuật toán.

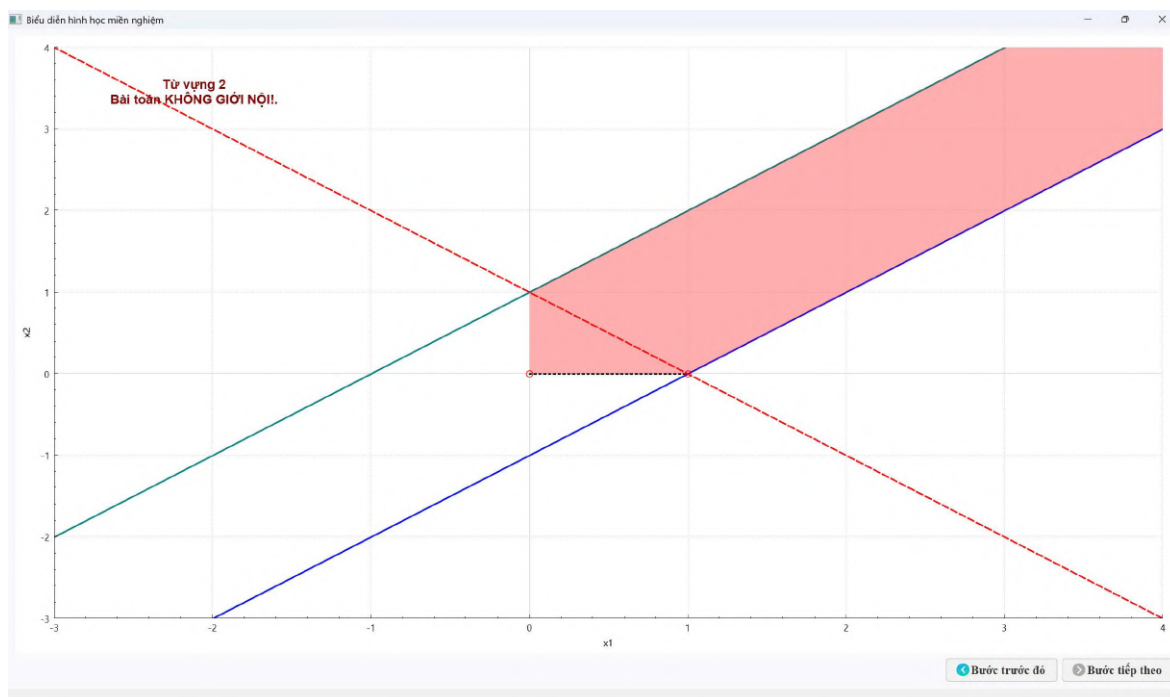
- **Giải thích không giới nội:** tồn tại một biến không cơ sở làm cải thiện hàm mục tiêu, đồng thời các hệ số của biến đó trong các phương trình ràng buộc không làm phá vỡ tính khả thi.
- **Giải thích:** Hàm mục tiêu có thể tăng lên vô hạn mà không vi phạm các ràng buộc. Do đó, giá trị tối ưu của bài toán Max Z là $+\infty$.

$$-Z = -1.00 - 2.00 x2 + 1.00 w1$$

$$x1 = 1.00 + 1.00 x2 - 1.00 w1$$

Biểu diễn hình học (Chỉ hỗ trợ cho trường hợp 2 biến số)
Hỏi/Đáp
Xem lời giải ở dạng PDF
OK

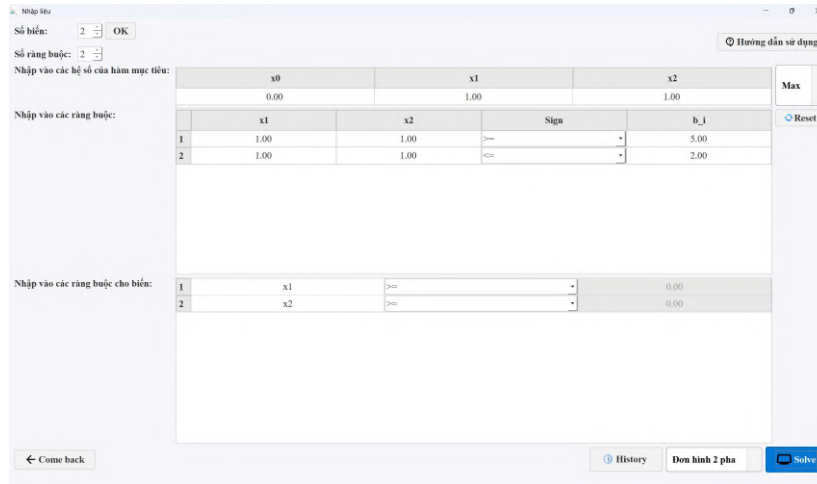
Hình 9.2.2: Nghiệm bài toán không giới nội



Hình 9.2.3: Biểu diễn hình học bài toán không giới nội

9.2.2 Trường hợp “Vô nghiệm”

Tập phương án của bài toán rỗng (các ràng buộc mâu thuẫn với nhau), không tồn tại lời giải thỏa mãn.



Số biến: 2 OK

Số ràng buộc: 2

Nhập vào các hệ số của hàm mục tiêu:

	x0	x1	x2
	0.00	1.00	1.00

Max

Reset

Hướng dẫn sử dụng

Nhập vào các ràng buộc:

	x1	x2	Sign	b_i
1	1.00	1.00	>=	5.00
2	1.00	1.00	<=	2.00

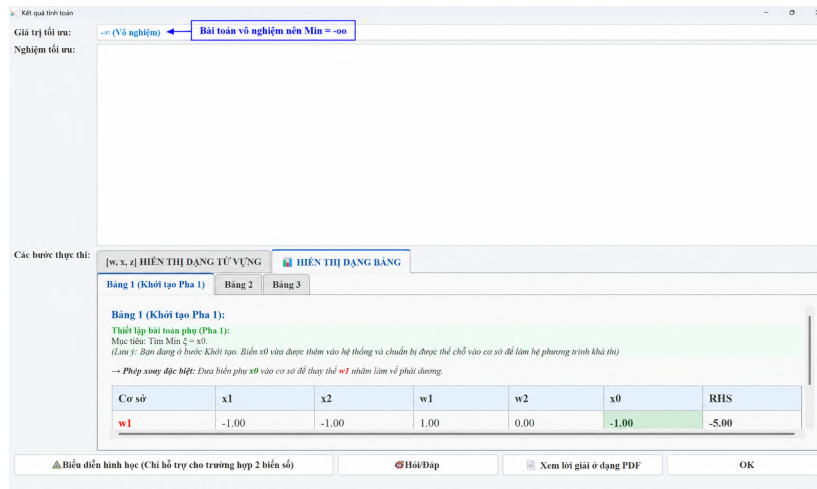
Nhập vào các ràng buộc cho biến:

	x1	Sign	b_i
1	x1	>=	0.00
2	x2	>=	0.00

← Come back

History Đơn hình 2 pha Solve

Hình 9.2.4: Bài toán vô nghiệm



Kết quả tính toán

Giá trị tối ưu: -∞ (Vô nghiệm) Bài toán vô nghiệm nên Min = -∞

Nghiệm tối ưu:

Các bước thực thi: [w, x, z] HIỂN THỊ DẠNG TỪ VỰNG HIỂN THỊ DẠNG BẢNG

Bảng 1 (Khởi tạo Pha 1) Bảng 2 Bảng 3

Bảng 1 (Khởi tạo Pha 1):

Thiết lập bài toán phụ (Pha 1):

Mục tiêu: Tìm Min $z = x_0$.

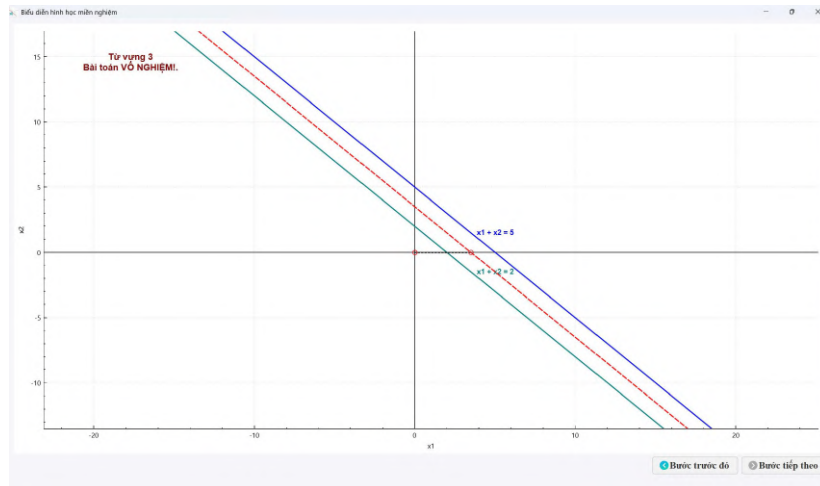
(Lưu ý: Bảng được tạo hoặc Khởi tạo. Biến số vào được thêm vào hệ thống và chuẩn bị được thể chế vào cơ sở để làm hệ phương trình khả thi)

→ Phương xoay đặc biệt: Dưa biến phụ x_0 vào cơ sở để thay thế w_1 nhằm làm vế phải dương.

Cơ sở	x1	x2	w1	w2	x0	RHS
w1	-1.00	-1.00	1.00	0.00	-1.00	-5.00

Δ Biểu diễn hình học (Chỉ hỗ trợ cho trường hợp 2 biến số) Hỏi/Đáp Xem lời giải ở dạng PDF OK

Hình 9.2.5: Nghiệm bài toán vô nghiệm



Hình 9.2.6: Biểu diễn hình học bài toán vô nghiệm

9.2.3 Trường hợp “Lỗi xoay vòng (Cycling)”

Xoay vòng là hiện tượng trong phương pháp đơn hình mà sau một số phép quay, thuật toán quay trở lại một cơ sở đã xuất hiện trước đó và tiếp tục lặp lại, nên thuật toán không dừng.

Cảnh báo Thuật toán

Hiện tượng xoay vòng (Cycling)!
Thuật toán bị lặp vô hạn.

OK

	x0	x1	x2	x3	x4	
	0.00	-10.00	57.00	9.00	24.00	Min
						Reset
	x1	x2	x3	x4	Sign	b_i
1	0.50	-5.50	-2.50	9.00	<=	0.00
2	0.50	-1.50	-0.50	1.00	<=	0.00
3	1.00	0.00	0.00	0.00	<=	0.00

	x1	x2	x3	x4	
1	x1	>=			0.00
2	x2	>=			0.00
3	x3	>=			0.00
4	x4	>=			0.00

← Come back

History Đơn hình Solve

Hình 9.2.7: Cảnh báo xoay vòng khi ta dùng thuật toán đơn hình để giải

Cách khắc phục: Ta sẽ chọn thuật toán Bland để giải bài toán xoay vòng này.

Kết quả tính toán

Giá trị tối ưu: -1.0000 → Đạt giá trị tối ưu sau 8 lần xoay

Nhiệm tối ưu:

Biến	Giá trị
x1	1.0000
x2	0.0000
x3	1.0000
x4	0.0000

Các bước thực thi:

[w, x, z] HIỂN THỊ DẠNG TỪ VỰNG HIỂN THỊ DẠNG BẢNG

Từ vựng 1 (Khởi tạo) Từ vựng 2 Từ vựng 3 Từ vựng 4 Từ vựng 5 Từ vựng 6 Từ vựng 7 Từ vựng 8

Từ vựng 1:
→ Chọn x1 làm biến vào, đẩy w1 ra khỏi cơ sở.

$$Z = 0.00 - 10.00 x1 + 57.00 x2 + 9.00 x3 + 24.00 x4$$

$$w1 = 0.00 - 0.50 x1 + 5.50 x2 + 2.50 x3 - 9.00 x4$$

$$w2 = 0.00 - 0.50 x1 + 1.50 x2 + 0.50 x3 - 1.00 x4$$

$$w3 = 1.00 - 1.00 x1$$

Δ Biểu diễn hình học (Chỉ hỗ trợ cho trường hợp 2 biến số) Hỏi/Đáp Xem lời giải ở dạng PDF OK

Hình 9.2.8: Từ vựng tối ưu thu được sau 8 lần xoay bằng thuật toán Bland

9.2.4 Trường hợp “Vô số nghiệm”

Bài toán đạt giá trị tối ưu tại nhiều điểm khác nhau.

Nhập tiêu

Số biến: 2 OK

Số ràng buộc: 1

Nhập vào các hệ số của hàm mục tiêu:

	x0	x1	x2
	0.00	1.00	1.00

Nhập vào các ràng buộc:

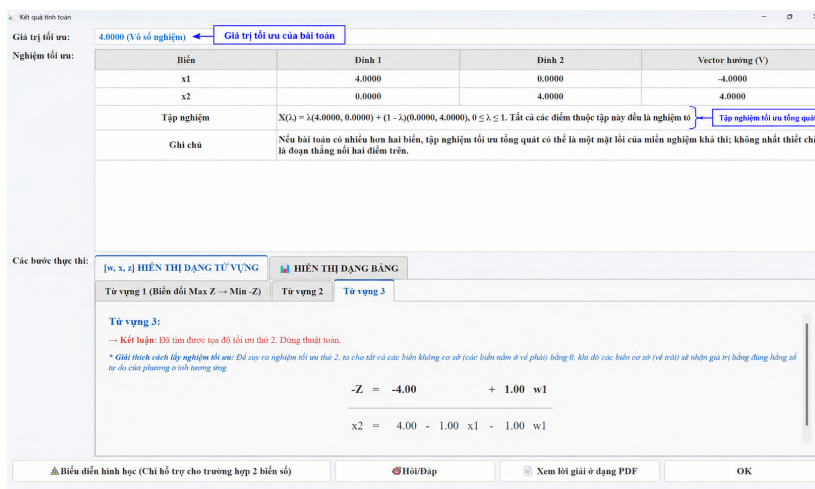
	x1	x2	Sign	b_i
1	1.00	1.00	<=	4.00

Nhập vào các ràng buộc cho biến:

	x1	Sign	b_i
1	x1	>=	0.00
2	x2	>=	0.00

← Come back History Bland Solve

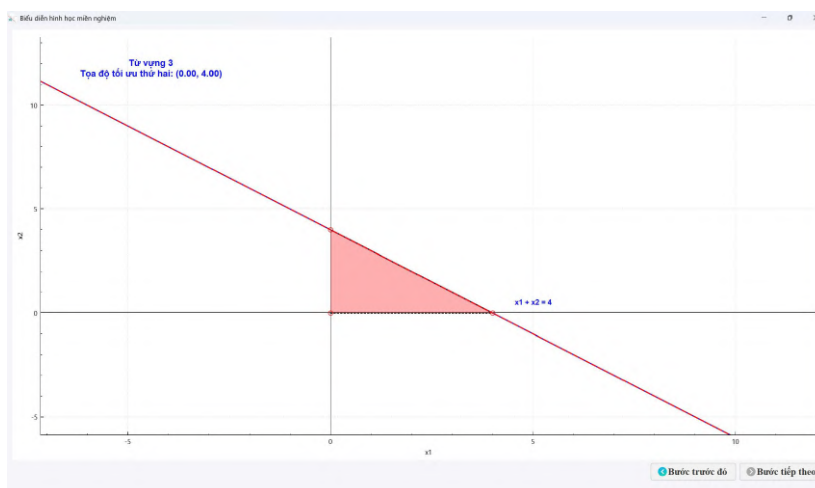
Hình 9.2.9: Bài toán Vô số nghiệm



Hình 9.2.10: Màn hình hiển thị kết quả bài toán vô số nghiệm



Hình 9.2.11: Biểu diễn hình học của bài toán vô số nghiệm


Hình 9.2.12: Hàm mục tiêu trùng với đoạn thẳng $x_1 + x_2 = 4$

9.3 Các tính năng hỗ trợ nhập liệu nâng cao

Ngoài việc nhập các số thực thông thường, phần mềm còn hỗ trợ nhập trực tiếp các biểu thức toán học vào ô hệ số. Hệ thống sẽ tự động tính toán và chuyển đổi biểu thức thành giá trị số tương ứng.

Người dùng có thể sử dụng các dạng biểu thức sau:

- **Các phép toán cơ bản**

Hỗ trợ các phép cộng, trừ, nhân và chia.

Ví dụ:

- $\pi - 1$

- $3/7$

- $2 * e$

- **Lũy thừa**

Sử dụng ký hiệu $\sim$ để tính lũy thừa.

Ví dụ:

- 2^3

- e^4

- π^4

- **Căn bậc hai và căn bậc n**

Hỗ trợ các hàm căn số thông dụng.

Ví dụ:

- $\text{sqrt}(2)$

- $\text{root3}(8)$ (Căn bậc ba của 8)

- **Biểu thức có ngoặc**

Cho phép kết hợp nhiều phép toán trong cùng một biểu thức.

Ví dụ:

- $(\pi - 1)/2$

- $-(2^4)$

- **Một số hằng số toán học được hỗ trợ**

- π : số $\pi \approx 3.141592654$

- e : số Euler $e \approx 2.718281828$

- **Lưu ý**

- Không sử dụng dấu phẩy (,) làm dấu thập phân.

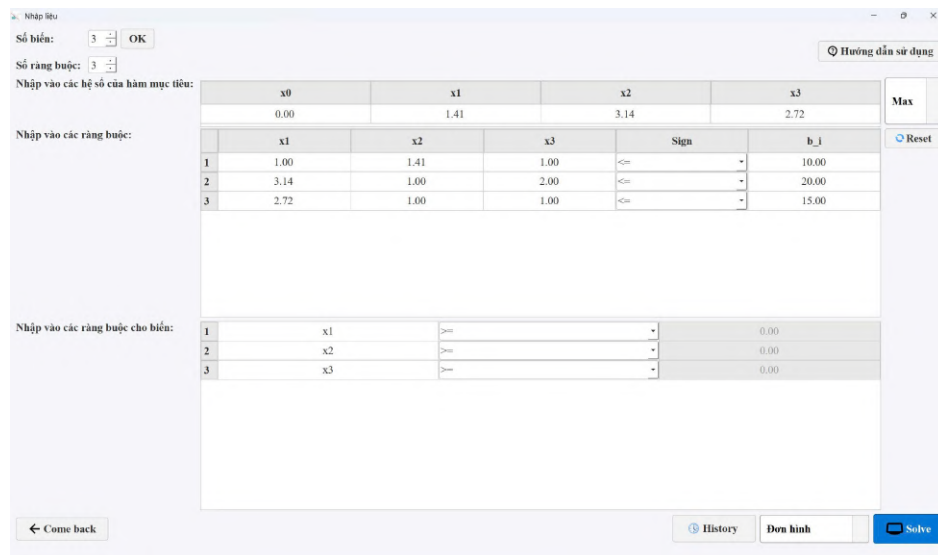
- Các dấu ngoặc phải được đóng đầy đủ và đúng cặp.

- Nên kiểm tra lại biểu thức trước khi nhấn **Solve**.

- Các giá trị như π , e , $\text{sqrt}(2)$, $\text{root3}(8)$ khi nhập vào phần mềm sẽ được tự động chuyển đổi thành số thập phân tương ứng.

Ví dụ:

$$\begin{aligned} \max \quad & Z = \sqrt{2}x_1 + \pi x_2 + ex_3 \\ \text{s.t.} \quad & x_1 + \sqrt{2}x_2 + x_3 \leq 10, \\ & \pi x_1 + x_2 + \sqrt[3]{8}x_3 \leq 20, \\ & ex_1 + x_2 + x_3 \leq 15, \\ & x_1, x_2, x_3 \geq 0. \end{aligned}$$



The screenshot shows a software window titled "Nhập tiêu" (Input Objective). It contains several input fields and tables for defining a linear programming problem.

Objective Function: The user has entered 3 variables. The objective function is defined as $Z = \sqrt{2}x_1 + \pi x_2 + ex_3$.

Constraints: The user has entered 3 constraints. The constraints are defined as follows:

	x0	x1	x2	x3	Sign	b_i
1	1.00	1.41	1.00	1.00	<=	10.00
2	3.14	1.00	2.00	1.00	<=	20.00
3	2.72	1.00	1.00	1.00	<=	15.00

Variable Bounds: The user has entered 3 variables. The bounds are defined as follows:

	x1	Sign	b_i
1	x1	>=	0.00
2	x2	>=	0.00
3	x3	>=	0.00

The interface includes buttons for "Come back", "History", "Đơn hình" (Simplex), and "Solve".

Hình 9.3.1: Các hằng số toán học sẽ tự động được chuyển đổi thành giá trị số tương ứng

10. Link mã nguồn của dự án và một vài mô tả về Repository

10.1 Link chứa toàn bộ mã nguồn của dự án

Toàn bộ mã nguồn của dự án được lưu trữ tại: <https://github.com/Alee-deg/SolveLinearProgramming>

10.2 Ghi chú tổng quan về repository

Phần thuật toán quy hoạch tuyến tính nằm trong thư mục `src/core`. Đây là phần quan trọng nhất khi đánh giá logic giải bài toán, bao gồm chuẩn hóa mô hình, xử lý ràng buộc, chọn biến vào/ra, cập nhật từ vệt/bảng đơn hình và phát hiện các trường hợp đặc biệt.

Phần giao diện người dùng nằm trong `src/windows` và `ui`. Các file này phụ trách màn hình nhập liệu, màn hình kết quả, biểu diễn hình học, lịch sử bài toán, xuất báo cáo và chatbot hỗ trợ giải thích.

Tài nguyên như logo, icon, hình ảnh và file đóng gói giao diện được đặt trong `resources`. Tài liệu hướng dẫn sử dụng được đặt trong `docs`. Cấu hình GitHub Actions để build tự động nằm trong `.github/workflows`.

10.3 Sơ đồ thư mục chính

```
SolveLinearProgramming/  
|--- .github/  
|   L--- workflows/  
|       L--- Các file YAML build tự động cho Windows, Linux, macOS  
|--- docs/  
|   L--- Tài liệu hướng dẫn sử dụng phần mềm  
|--- resources/  
|   |--- resource.qrc  
|   |--- app_icon.rc  
|   |--- images/  
|       L--- Logo, icon, hình ảnh dùng trong giao diện  
|   L--- icons/  
|       L--- Các biểu tượng SVG/PNG hỗ trợ giao diện  
|--- src/  
|   |--- main.cpp  
|   |--- core/  
|       |--- simplex solver.h  
|       L--- simplex solver.cpp  
|   L--- windows/  
|       |--- mainwindow.cpp / mainwindow.h  
|       |--- dashboard.cpp / dashboard.h  
|       |--- wdsolve.cpp / wdsolve.h
```

```
|      |--- wdshowimage.cpp / wdshowimage.h
|      |--- wdchatbot.cpp / wdchatbot.h
|      L--- qcustomplot.cpp / qcustomplot.h
|--- ui/
|      |--- mainwindow.ui
|      |--- dashboard.ui
|      |--- wdsolve.ui
|      |--- wdshowimage.ui
|      L--- wdchatbot.ui
|--- CMakeLists.txt
|--- README.md
L--- .gitignore
```

10.4 Vai trò của từng nhóm thư mục/file

Thành phần	Vai trò	Gợi ý khi chấm
src/core/simplexsolver.cpp, simplexsolver.h	Chứa thuật toán giải quy hoạch tuyến tính: đơn hình, hai pha, xử lý nghiệm tối ưu, vô nghiệm, vô số nghiệm và không giới nội.	Ưu tiên kiểm tra phần này để đánh giá đúng trọng tâm thuật toán.
src/windows/wdsolve.cpp	Hiển thị kết quả tính toán, các bước giải dạng từ vựng/bảng, giải thích nghiệm và xuất báo cáo PDF/LaTeX.	Kiểm tra cách trình bày lời giải, giá trị tối ưu, tập nghiệm và báo cáo xuất ra.
src/windows/dashboard.cpp	Màn hình nhập liệu, lịch sử bài toán, tải lại bài cũ, ghi chú và điều hướng sang các cửa sổ chức năng.	Kiểm tra nhập số biến, số ràng buộc, lưu/tải lịch sử.
src/windows/wdshowimage.cpp	Biểu diễn hình học miền nghiệm cho bài toán 2 biến.	Kiểm tra với bài toán 2 biến có nghiệm duy nhất, vô số nghiệm, ràng buộc bằng.
src/windows/wdchatbot.cpp	Chức năng hỏi/đáp hỗ trợ giải thích bài toán, có cấu hình API key.	Có thể kiểm tra phụ, không phải trọng tâm thuật toán.
ui/*.ui	Các file giao diện thiết kế bằng Qt Designer.	Giúp xem bố cục màn hình nếu cần chỉnh giao diện.
resources/	Chứa logo, icon, hình ảnh và resource.qrc.	Kiểm tra khi build nếu thiếu icon hoặc tài nguyên.

Thành phần	Vai trò	Gợi ý khi chấm
<code>.github/workflows/</code>	Cấu hình build tự động và đóng gói cho nhiều hệ điều hành.	Kiểm tra khả năng đóng gói sản phẩm.
<code>docs/</code>	Tài liệu hướng dẫn sử dụng phần mềm.	Dùng để xem hướng dẫn cài đặt, nhập bài toán và chạy thử.
<code>CMakeLists.txt</code>	Cấu hình build dự án bằng CMake.	Kiểm tra đường dẫn source, UI, resource và thư viện Qt.

10.5 Ghi chú cho giảng viên khi xem mã nguồn

- Dự án không đặt toàn bộ mã nguồn trong một file mà chia theo chức năng để dễ bảo trì và dễ chấm.
- Khi muốn kiểm tra tính đúng đắn của thuật toán, nên tập trung vào `src/core` trước, sau đó đối chiếu với phần hiển thị trong `src/windows/wdsolve.cpp`.
- Khi muốn kiểm tra giao diện và trải nghiệm người dùng, nên mở `dashboard.cpp`, `wdsolve.cpp`, `wdshowimage.cpp` và các file `.ui` tương ứng.
- Khi muốn kiểm tra khả năng đóng gói sản phẩm, xem `CMakeLists.txt` và thư mục `.github/workflows`.
- Khi muốn chạy thử nhanh, có thể tải bản build trong Releases hoặc build từ mã nguồn bằng CMake/Qt.

11. Một số phần mềm giải quy hoạch tuyến tính khác

- [1] Microsoft Excel với công cụ *Solver*
- [2] Phần mềm giải toán Quy hoạch tuyến tính, *Tran Manh Han*, link phần mềm: <https://groups.google.com/g/tranmanhhhan/c/AMuhLKsFAJk>
- [3] PHPSimplex, *Daniel Izquierdo Granja, Juan José Ruiz Ruiz*, link: <https://www.phpsimplex.com/en/index.htm#>

12. Vai trò của các thành viên trong dự án

Đây là bảng đánh giá vai trò và hiệu suất của các thành viên của nhóm trong suốt quá trình làm bài tập lớn lần này:

STT	MSSV	Họ và tên	Email	Nhiệm vụ	Đánh giá
1	24110158	Lê Hoài Hậu	24110158@student.hcmus.edu.vn	Nhóm trưởng, thiết kế thuật toán, lập trình phần mềm, đóng gói sản phẩm, phần 8, 10 trong báo cáo	100%
2	24110149	Phạm Trần Khánh Duy	24110149@student.hcmus.edu.vn	Đánh LaTeX phần lớn báo cáo, kiểm thử phần mềm, đóng góp ý kiến cải thiện giao diện và thuật toán, các phần 3, 4 trong báo cáo	100%
3	24110153	Phạm Ngọc Hải	24110153@student.hcmus.edu.vn	Kiểm thử phần mềm, đóng góp ý kiến cải thiện giao diện/thuật toán, các phần 1, 2, 5, 6, 7 trong báo cáo	100%
4	24110137	Nguyễn Quốc Đạt	24110137@student.hcmus.edu.vn	Thiết kế thuật toán, Đánh LaTeX phần 9 của báo cáo, đóng góp ý kiến cải thiện giao diện và thuật toán, phần 9 trong báo cáo, kiểm thử phần mềm	100%
5	24110191	Ngô Nguyễn Huy Khương	24110191@student.hcmus.edu.vn	Thiết kế thuật toán, đóng góp ý kiến cải thiện giao diện, thuật toán phần 9 trong báo cáo (phụ), viết phần giới thiệu, chính sách trong phần mềm	100%

Bảng 12.0.1: Bảng phân công nhiệm vụ và đánh giá thành viên